Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

Introduction

Course Website

https://cs230-iitb.github.io/autumn-2025/

Who is talking??

- Sayandeep Saha (https://www.cse.iitb.ac.in/~sayandeepsaha/)
  - Research Interests: Security and Cryptography
  - Research Group: SHArC (https://sharc.vercel.app/)
  - Office Hours: Monday 10:30 am — 11:00 am after the class/ Through prior appointment

I am a Computer Scientist…

CS230 Deep Learning

Deep Learning is one of the most highly sought after skills in AI. In this course, you will learn the foundations of Deep Learning, understand how to build neural networks, and learn how to lead successful machine learning projects. You will learn about Convolutional networks, RNNs, LSTM, Adam, Dropout, BatchNorm, Xavier/He initialization, and more.

Syllabus

Ed (link via Canvas)

Lecture videos (Canvas)

Lecture videos (Fall 2018)

Instructors

Andrew Ng

Instructor

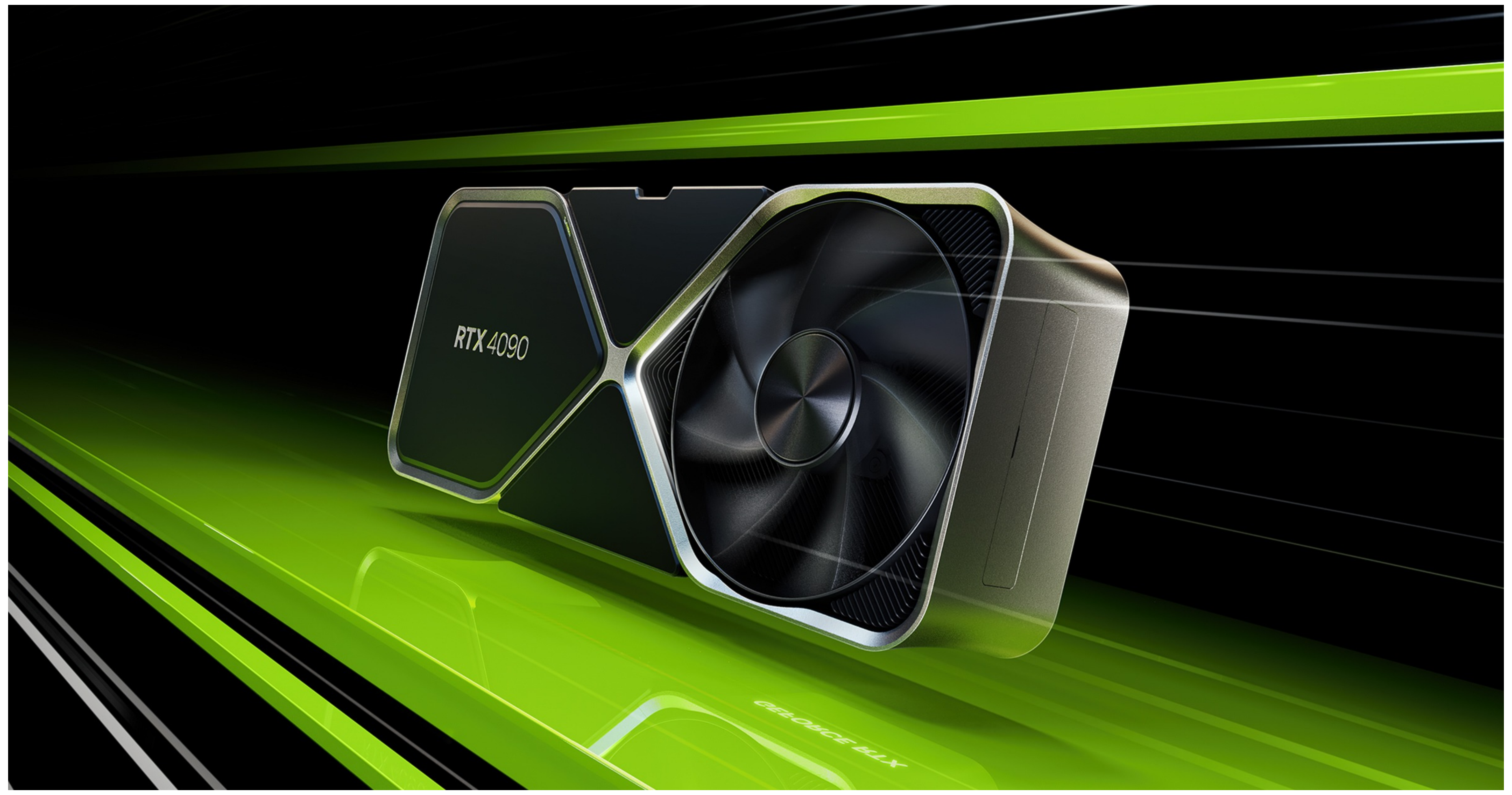
Time and Location

Wednesday 9:30AM-11:20AM

Zoom

Sorry Guys… But this is also about…

- Machine and Learning…Better to say “learning about machines…”

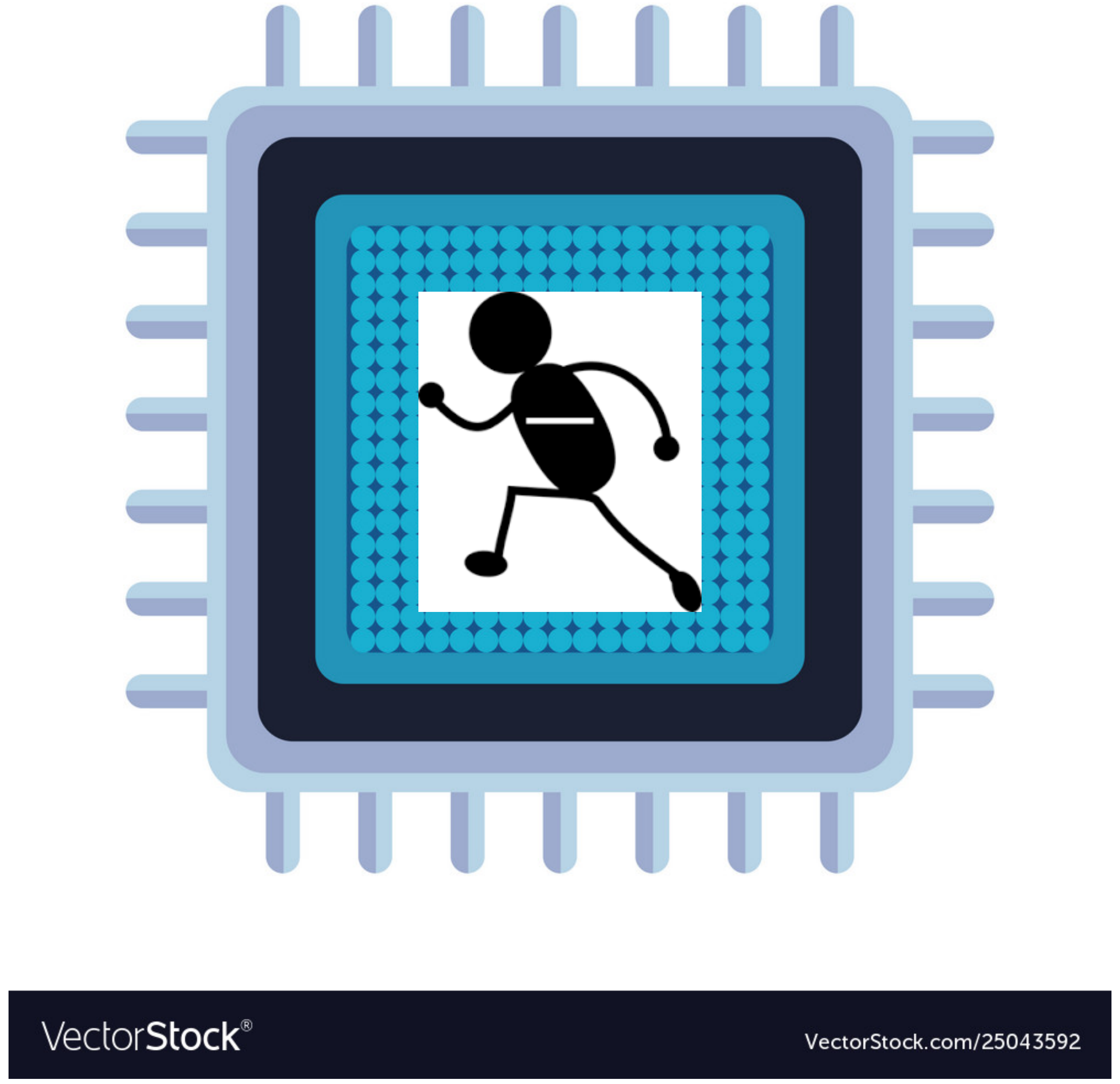


How is it Related to “Machine Learning”?

- **Ever though about where your tensorflow/pytorch runs?**
- Machine Learning
  - Concepts were there since 1980’s
  - Then why such buzz in 2025??
- Those days we didn’t have such powerful machines to train our models

Why should I bother about chips??

- **It is Electronics… :(**
- Indeed it is….But then…



Why should I bother about chips??

- **It is Electronics… :(**
- Indeed it is….But then everyone wants their own processor, why???

Google's Next Pixel Phone Will Be Powered by a Custom Chip

Following the industry trend of tech giants manufacturing their own processors, the company will start putting bespoke silicon in its mobile hardware.

News / Technology Science

Meta Launches New AI Chip For Facebook, Instagram And WhatsApp

The chip, internally known as "Artemis," signifies Meta's move towards reducing reliance on Nvidia's AI chips and cutting down energy costs.

TIMES NOW **Authored by:** TN Tech Desk | Updated Apr 11, 2024, 09:50 IST

Why should I bother about chips??

- **It is Electronics… :(**
- Hardware is the new software

Google's Next Pixel Phone Will Be Powered by a Custom Chip

Following the industry trend of tech giants manufacturing their own processors, the company will start putting bespoke silicon in its mobile hardware.

News / Technology Science

Meta Launches New AI Chip For Facebook, Instagram And WhatsApp

The chip, internally known as "Artemis," signifies Meta's move towards reducing reliance on Nvidia's AI chips and cutting down energy costs.

Authored by: TN Tech Desk | Updated Apr 11, 2024, 09:50 IST

Abstraction…

- **We only use our computers…do not bother what is inside…**
  - Don’t bother about unnecessary complexity…just use things as black box
  - Can you develop a game while thinking on how your electrons behave??

Abstraction…

Problems

Algorithms

Programming Languages/Compilers

Operating/Database/Network Systems

Architecture
Digital gates (below that: electrons ☺)

https://www.cse.iitb.ac.in/~biswa/courses/CS230/lectures/L1.pdf

Abstraction…

- **But then, it only works when you don't care much about performance…**
  - Recent trend in industry…
    - Design and optimize everything…
  - Breaking the abstraction barrier..
    - But that's ok, performance gives money

And Finally, Security…

- **Hardware hacking is the new "cool" thing…**

Intel ups protection against physical chip attacks in Alder Lake

Repurposes logic originally used for spotting variations in voltage, timing in older circuits to help performance

Dan Robinson Fri 12 Aug 2022 15:00 UTC

BLACK HAT Intel has disclosed how it may be able to protect systems against some physical threats by repurposing circuitry originally designed to counter variations in voltage and timing that may occur as silicon circuits age.

The research was presented by Intel at the Black Hat USA 2022 cybersecurity conference this week, and details logic inside the system chipset that is intended to complement existing software mitigations for fault injection attacks, the chipmaker said.

It makes use of a Tunable Replica Circuit (TRC), logic developed at Intel Labs to monitor variations such as voltage droop, temperature, and aging in circuits to improve

TRENDING: Maingear MG-1 AMD PC Build GeForce RTX 4070 Radeon RX 7900 XTX Ryzen 7 7800X3D Alienw

HOME NEWS

AMD And Intel CPUs Rocked By New Speculative Execution Attack With A Huge Performance Hit

by Zak Killian — Wednesday, July 13, 2022, 05:08 PM EDT

0 Comments BECOME A PATRON

Crypto Keys Could Be Compromised by Intel and AMD 'Hertzbleed' Chip Vulnerability

2 mins

By Martin Young 15 June 2022, 07:40 GMT+0000

Updated by Geraint Price 15 June 2022, 07:40 GMT+0000

In Brief

The attack observes power signature of any cryptographic workload.

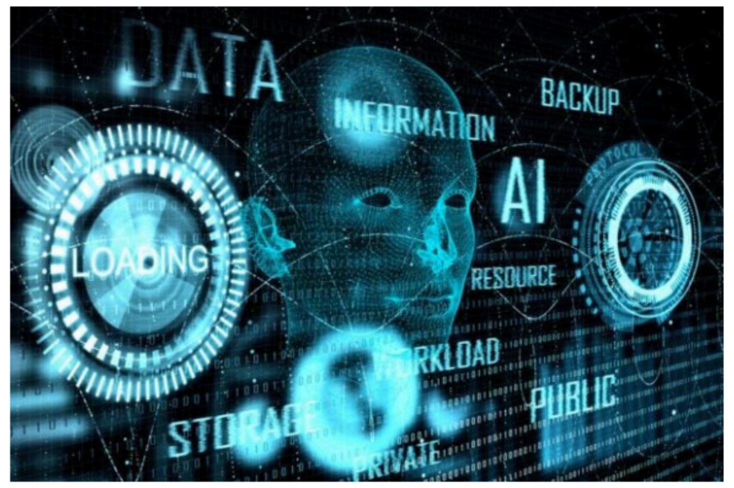
Modern Intel Core and AMD Ryzen chips are affected.

We use cookies to improve your experience

Technology

Using just a $25 device a researcher hacked into Elon Musk's Starlink system

What will the technology mogul have to say about this?



AI power analysis breaks post-quantum security algorithm

Technology News | February 19, 2023

By Nick Flaherty

AUTHENTICATION & ENCRYPTION

One of the key post quantum security algorithms agreed as the latest standard has been broken by Swedish researchers

And Finally, Security…

- **Hardware hacking is the new “cool” thing…**
  - We can just hack a hardware by writing a piece of code


Intel ups protection against physical chip attacks in Alder Lake

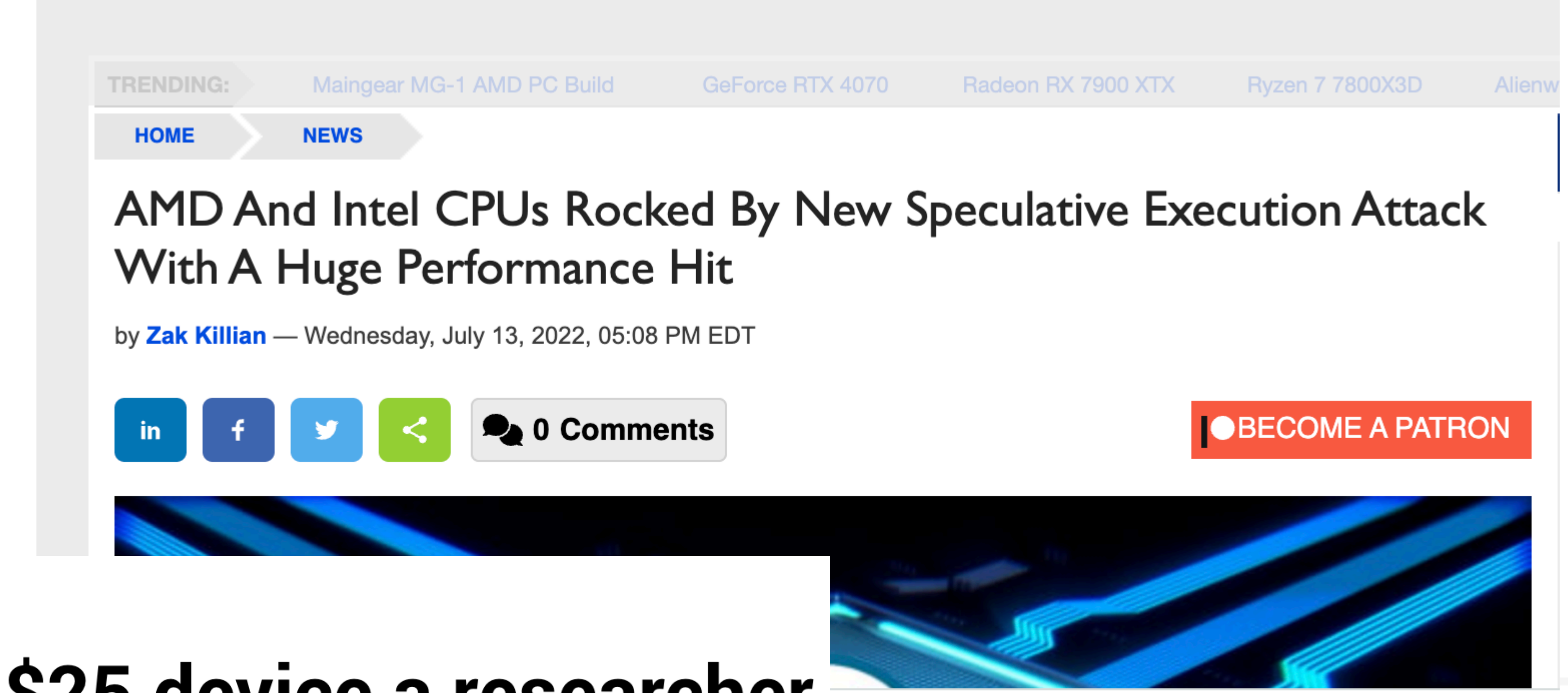
Repurposes logic originally used for spotting variations in voltage, timing in older circuits to help performance

Dan Robinson — Fri 12 Aug 2022 / 15:00 UTC

BLACK HAT Intel has disclosed how it may be able to protect systems against some physical threats by repurposing circuitry originally designed to counter variations in voltage and timing that may occur as silicon circuits age.

The research was presented by Intel at the Black Hat USA 2022 cybersecurity conference this week, and details logic inside the system chipset that is intended to complement existing software mitigations for fault injection attacks, the chipmaker said.

It makes use of a Tunable Replica Circuit (TRC), logic developed at Intel Labs to monitor variations such as voltage droop, temperature, and aging in circuits to improve

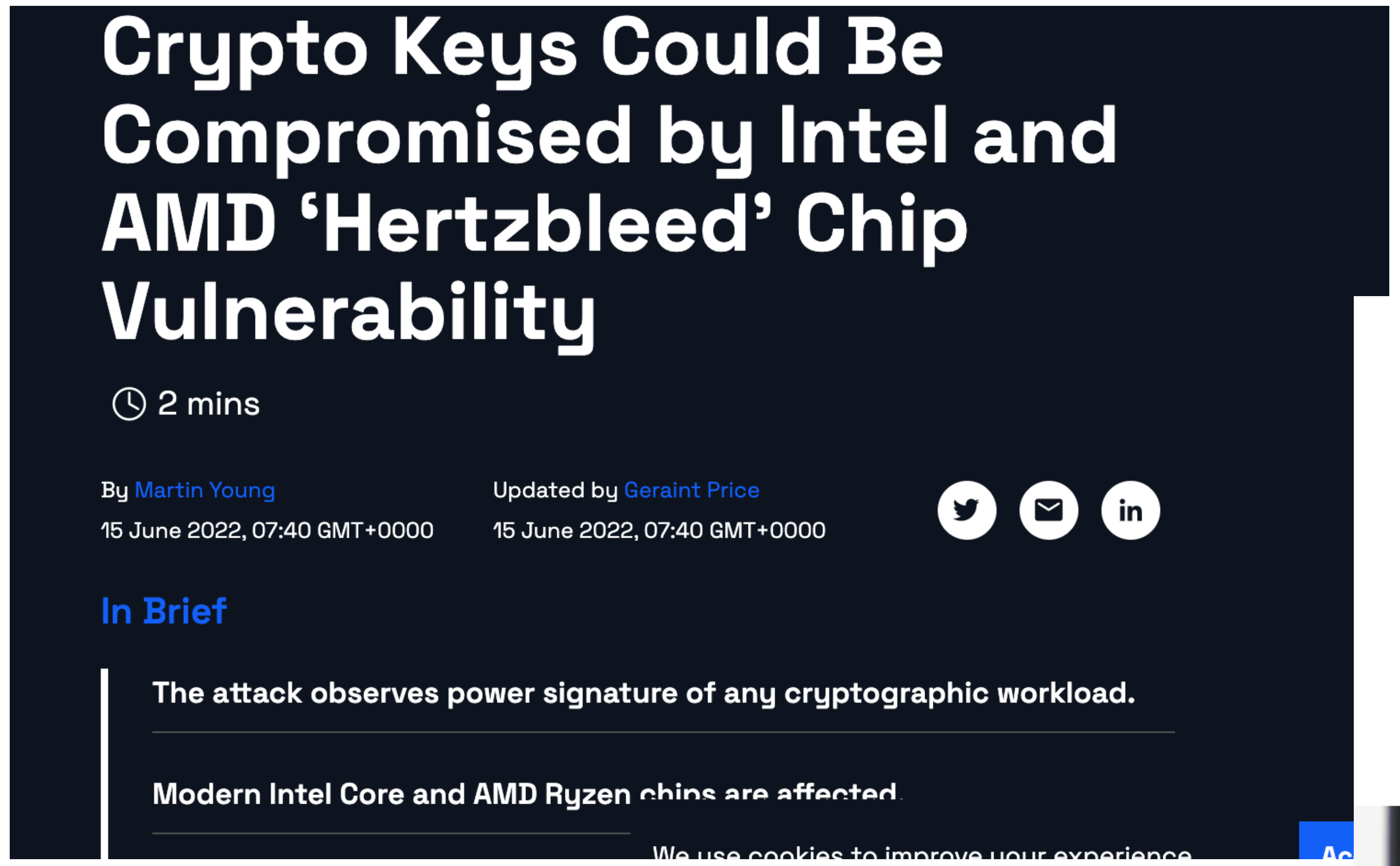

TRENDING: Maingear MG-1 AMD PC Build GeForce RTX 4070 Radeon RX 7900 XTX Ryzen 7 7800X3D Alienw

HOME NEWS

AMD And Intel CPUs Rocked By New Speculative Execution Attack With A Huge Performance Hit

by Zak Killian — Wednesday, July 13, 2022, 05:08 PM EDT

0 Comments BECOME A PATRON


Crypto Keys Could Be Compromised by Intel and AMD ‘Hertzbleed’ Chip Vulnerability

2 mins

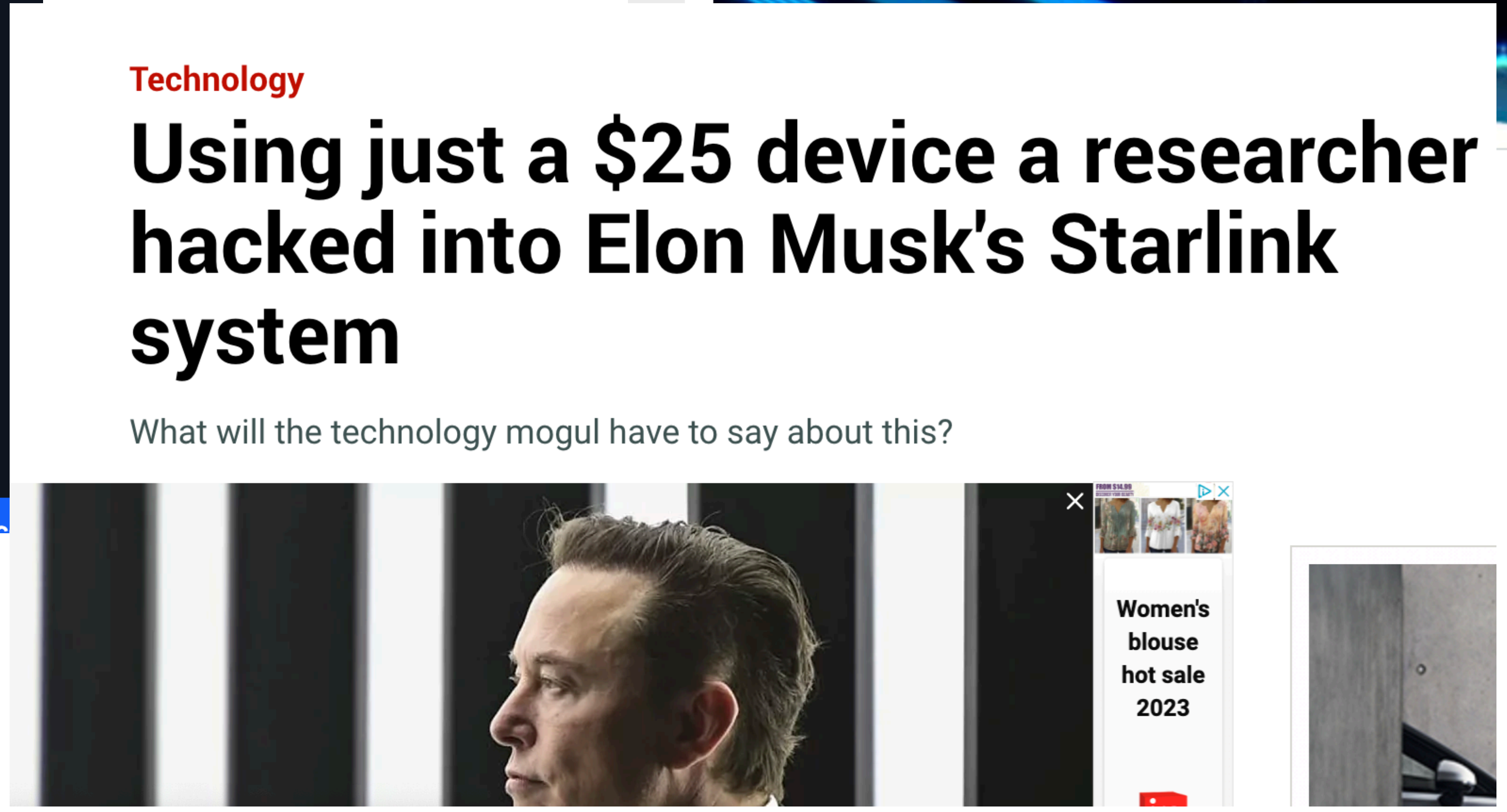
By Martin Young 15 June 2022, 07:40 GMT+0000

Updated by Geraint Price 15 June 2022, 07:40 GMT+0000

In Brief

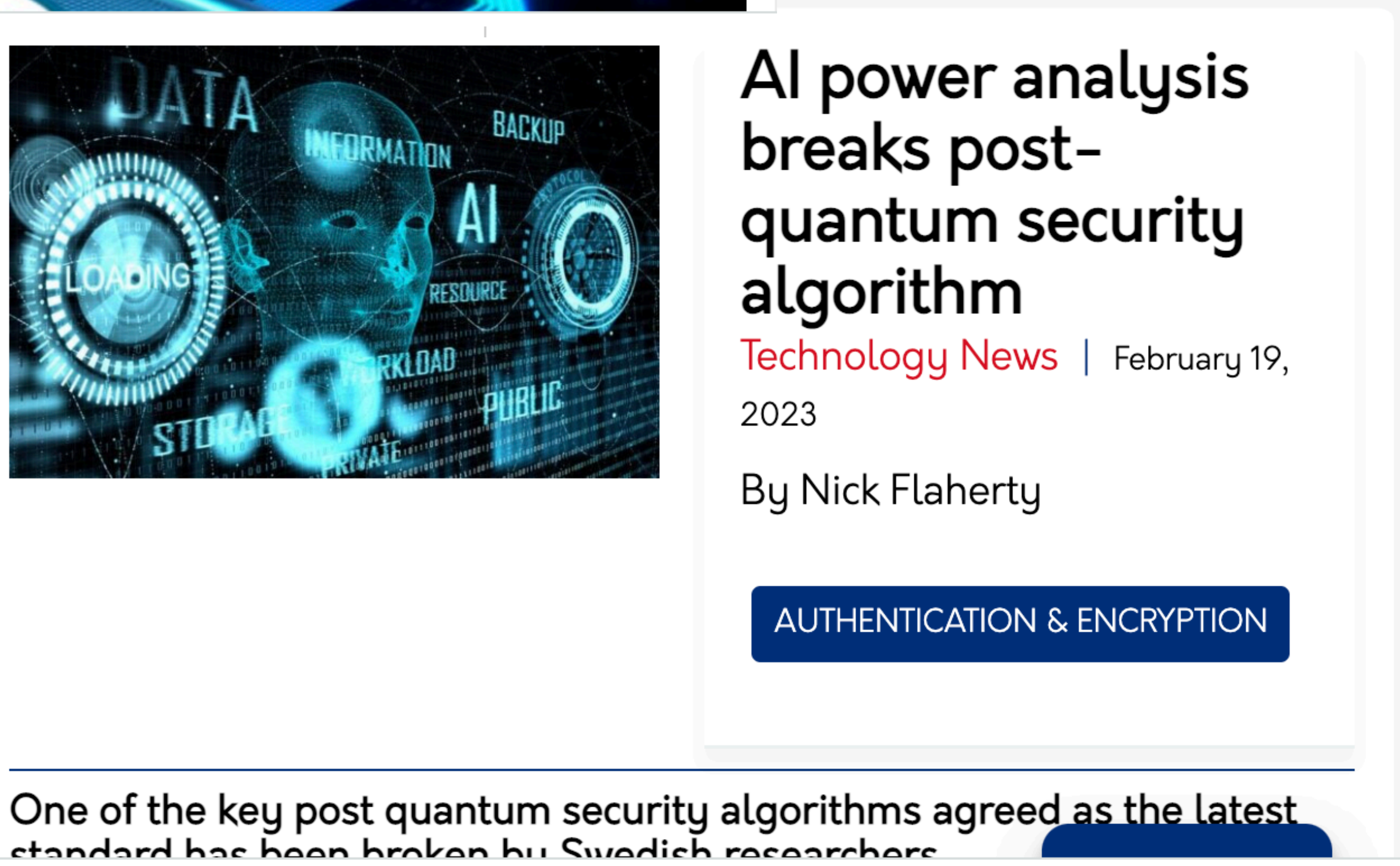
The attack observes power signature of any cryptographic workload.

Modern Intel Core and AMD Ryzen chips are affected.


Technology

Using just a $25 device a researcher hacked into Elon Musk's Starlink system

What will the technology mogul have to say about this?



AI power analysis breaks post-quantum security algorithm

Technology News | February 19, 2023

By Nick Flaherty

AUTHENTICATION & ENCRYPTION

One of the key post quantum security algorithms agreed as the latest standard has been broken by Swedish researchers

And Finally, Security…

- **Hardware hacking is the new “cool” thing…**
- We can just hack a hardware by writing a piece of code

Intel ups protection against physical chip attacks in Alder Lake

Repurposes logic originally used for spotting variations in voltage, timing in older circuits to help performance

Dan Robinson — Fri 12 Aug 2022 // 15:00 UTC

BLACK HAT Intel has disclosed how it may be able to protect systems against some physical threats by repurposing circuitry originally designed to counter variations in voltage and timing that may occur as silicon circuits age.

The research was presented by Intel at the Black Hat USA 2022 cybersecurity conference this week, and details logic inside the system chipset that is intended to complement existing software mitigations for fault injection attacks, the chipmaker said.

It makes use of a Tunable Replica Circuit (TRC), logic developed at Intel Labs to monitor variations such as voltage droop, temperature, and aging in circuits to improve

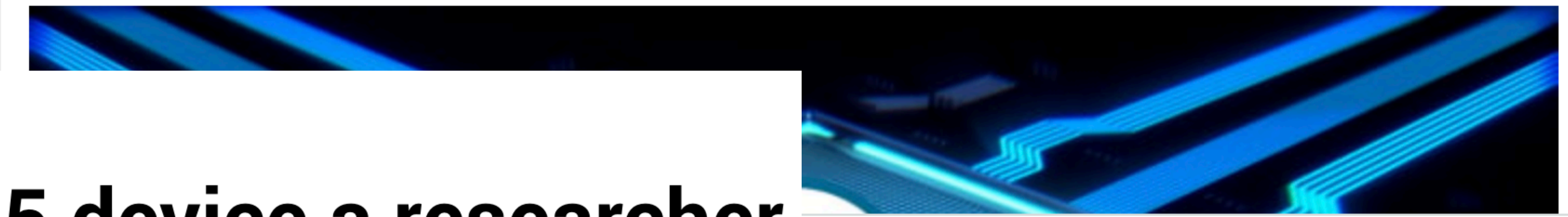
TRENDING: Maingear MG-1 AMD PC Build GeForce RTX 4070 Radeon RX 7900 XTX Ryzen 7 7800X3D Alienw

HOME NEWS

AMD And Intel CPUs Rocked By New Speculative Execution Attack With A Huge Performance Hit

by Zak Killian — Wednesday, July 13, 2022, 05:08 PM EDT

0 Comments BECOME A PATRON



Crypto Keys Could Be Compromised by Intel and AMD ‘Hertzbleed’ Chip Vulnerability

2 mins

By Martin Young 15 June 2022, 07:40 GMT+0000

Updated by Geraint Price 15 June 2022, 07:40 GMT+0000

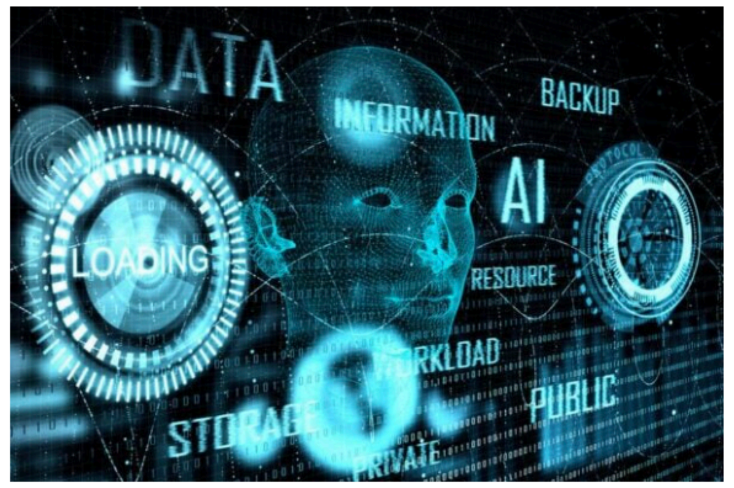
In Brief

- The attack observes power signature of any cryptographic workload.
- Modern Intel Core and AMD Ryzen chips are affected.

Technology

Using just a $25 device a researcher hacked into Elon Musk’s Starlink system

What will the technology mogul have to say about this?



AI power analysis breaks post-quantum security algorithm

Technology News | February 19, 2023

By Nick Flaherty

AUTHENTICATION & ENCRYPTION

One of the key post quantum security algorithms agreed as the latest standard has been broken by Swedish researchers

Let’s Pause

- **This is a foundational course for entering…**
  - The world on modern computer architecture, AI accelerators…
    - **Byproduct**: It will make you a better coder for sure…
  - Modern computer security
  - And many more…
- Let’s now talk a bit on the course logistics…

Logestics

Plan Ahead…

Mid Semester Exam	30%
End Semester Exam	40%
Quiz (two)	30% (combined)
Tutorials	Not Graded
Home Assignments	Not Graded

Plan Ahead…

Tutorial I (tentative)	2nd/3rd week of Aug
Tutorial II (tentative)	1st/2nd week of Sept
Quiz I (tentative)	1st/2nd week of Sept
Mid-Sem	3rd week of Sept
Tutorial III (tentative)	2nd week of Oct
Tutorial IV (tentative)	4th week Oct/1st week Nov
Quiz II (tentative)	4th week Oct/1st week Nov
Last Class	7th Nov

The tutorial and quiz dates will be **solely decided by the instructor** and will be announced one week before.

Do no ask for any adjustments in these dates. Anyone asking will lose 2 marks

Attendance…

No attendance for class

Attendance is Mandatory for Tutorials and Quizzes

Either come or not come. Do not enter class if you more than 5 mins late

Slides…

Slides will not contain everything

Attend lectures, Take class notes.

Pause me and ask **Questions**

What I Expect…

Please ask questions…

I will be more than happy if you are interested to do some advanced project — not as a part of these course though

CS231

No lab on the first week

Will be taken by Professor Biswa

Labs might not finish in 3 hours, take home, finish at your own convenience

Academic Dishonesty

https://www.iitb.ac.in/newacadhome/procedures201521July.pdf

https://www.iitb.ac.in/newacadhome/punishments201521July.pdf

Academic Dishonesty

https://www.iitb.ac.in/newacadhome/procedures201521July.pdf

https://www.iitb.ac.in/newacadhome/punishments201521July.pdf

Piazza

All notifications on Piazza only

https://piazza.com/iit_bombay/summer2025/cs230

Access Code: cs230

Final grades will be on Moodle

Course Website

https://cs230-iitb.github.io/autumn-2025/

TA Interaction

There will be group-wise TAs

Your group TA will be your first point of contact

How to Reach Me?

- Mailto: sayandeepsaha@cse.iitb.ac.in
  - Sub: [CS230]: Group ID: <your sub>
- Always CC to your group TA.
- Please don't mail to sayandeepsaha@iitb.ac.in OR sayandeep@cse.iitb.ac.in
- Please do not spam

TAs

To be announced this week…

We shall make the groups and assign TAs for that group

Absence in Exams

- Absence in Exams:
  - Mail with proper provable justification (e.g., institute-approved documents)
  - Mail by CCing the TA
  - Mail ASAP (preferably within a day of exam)

Crib Sessions During Exam…

Handle it carefully…

Do not mail, everything should be through the crib forms etc..

Lecture Formats

- Traditional — no flipped classroom
- Interactive
- Stop and ask me if you don't get it.
- Do not assume that you will get all from slides — slides are just summaries.
- You take care of your study, we shall take care of your grades

Finally…

No open screen in my class. You should leave if you have urgent calls or WhatsApp messages to reply

Thank you

Digital Logic Design + Computer Architecture

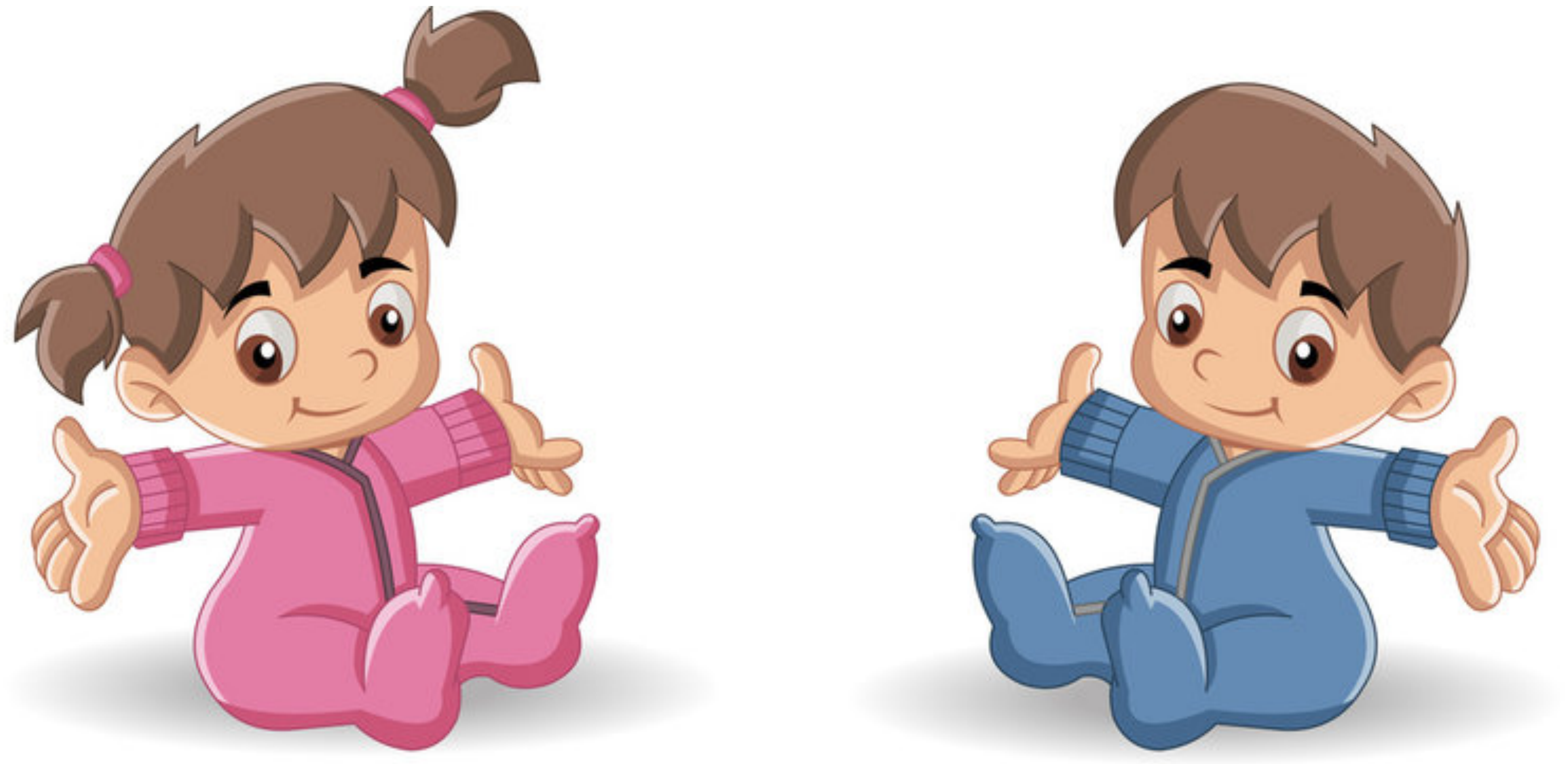
Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

Number Systems and Codes

Baby Step

- **Let’s go back to the days when we were 2 years old…**
    - Learning numbers again….but in a new way…



Numbers in Computing

- We normally use the decimal number system.
- But computers understand only bits…
- How to compute on bits??

Generalization of Number Representation

- Numbers are represented in a "base".
- Decimal (Base 10): $$953.78_{10} = 9 \times 10^2 + 5 \times 10^1 + 3 \times 10^0 + 7 \times 10^{-1} + 8 \times 10^{-2}$$
- Binary (Base 2): $$\begin{aligned} 1011.11_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 8 + 0 + 2 + 1 + \tfrac{1}{2} + \tfrac{1}{4} = 11\tfrac{3}{4} = 11.75_{10} \end{aligned}$$
- The general case — base b number:
$(N)_b = a_{q-1}b^{(q-1)} + a_{q-2}b^{(q-2)} + \cdots + a_2b^2 + a_1b^1 + a_0b^0 + a_{-1}b^{-1} + \cdots a_{-p}b^{-p}, \quad 0 \le a_i < b, b > 1$
  - $a_{(q-1)}$ is called the Most Significant Digit (MSD)
  - $a_{(-p)}$ is called the Least Significant Digit (MSD)
  - $a_{(-1)}$ — $a_{(-p)}$ are digits in the fractional part.

Generalization of Number Representation

Base				
2	4	8	10	12
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	10	4	4	4
0101	11	5	5	5
0110	12	6	6	6
0111	13	7	7	7
1000	20	10	8	8
1001	21	11	9	9
1010	22	12	10	α
1011	23	13	11	β
1100	30	14	12	10
1101	31	15	13	11
1110	32	16	14	12
1111	33	17	15	13

Base Conversion

- Octal (b=8) to Decimal (b=10): $(432.2)_8 = 4 \times 8^2 + 3 \times 8^1 + 2 \times 8^0 + 2 \times 8^{-1} = (282.25)_{10}$
- Binary (base 2) to Decimal (b=10): $(1010.011)_2 = 2^3 + 2^1 + 2^{-2} + 2^{-3} = (10.375)_{10}$

Base Conversion: Decimal to…

To Binary:

$2\,/\,53$

$2\,/\,26 \quad \text{rem.} = 1 = a_0$

$2\,/\,13 \quad \text{rem.} = 0 = a_1$

$2\,/\,6 \quad \text{rem.} = 1 = a_2 \qquad 53_{10} = 110101_2$

$2\,/\,3 \quad \text{rem.} = 0 = a_3$

$2\,/\,1 \quad \text{rem.} = 1 = a_4$

$0 \quad \text{rem.} = 1 = a_5$

Convenient way of writing

Integer	Remainder	
41		
20	1	
10	0	
5	0	
2	1	
1	0	
0	1	101001 = answer

To Octal:

153		
19	1	
2	3	
0	2	$= (231)_8$

Base Conversion

- General Rule: for $b_1 > b_2$

$$(N)_{b_1} = a_{q-1}b_2^{q-1} + a_{q-2}b_2^{q-2} + \cdots + a_1 b_2^1 + a_0 b_2^0$$

$$\frac{(N)_{b_1}}{b_2} = \underbrace{a_{q-1}b_2^{q-2} + a_{q-2}b_2^{q-3} + \cdots + a_1}_{Q_0} + \frac{a_0}{b_2}$$

$$\left(\frac{Q_0}{b_2}\right)_{b_1} = \underbrace{a_{q-1}b_2^{q-3} + a_{q-2}b_2^{q-4} + \cdots}_{Q_1} + \frac{a_1}{b_2}$$

Base Conversion: Fractions

To Binary:

	Integer		Fraction	Coefficient
$0.6875 \times 2 =$	1	+	0.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0	+	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1	+	0.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1	+	0.0000	$a_{-4} = 1$

Answer: $(0.6875)_{10} = (0.a_{-1}a_{-2}a_{-3}a_{-4})_2 = \underline{(0.1011)_2}$

To Octal:

$$0.513 \times 8 = 4.104$$
$$0.104 \times 8 = 0.832$$
$$0.832 \times 8 = 6.656$$
$$0.656 \times 8 = 5.248$$
$$0.248 \times 8 = 1.984$$
$$0.984 \times 8 = 7.872$$

$$(0.513)_{10} = (0.406517\ldots)_8$$

$$\boxed{\begin{aligned}(N)_{b_1} &= a_{-1}b_2^{-1} + a_{-2}b_2^{-2} + \cdots + a_{-p}b_2^{-p} \\ b_2 \cdot (N)_{b_1} &= a_{-1} + a_{-2}b_2^{-1} + \cdots + a_{-p}b_2^{-p+1}\end{aligned}}$$

- Important!!! If a number has both integer and fraction parts, then conversion of these two parts are done separately.
- Simplest way of converting from base b_1 to b_2, is to convert from b_1 to base 10 and then base 10 base b_2.

Quiz Time

- Convert $(231.3)_4$ to base 7

Quiz Time

- Convert $(231.3)_4$ to base 7
- <u>Ans:</u>

$$(231.3)_4 = 2 \times 4^2 + 3 \times 4^1 + 1 \times 4^0 + 3 \times 4^{-1} = (45.75)_{10}$$

$$\begin{array}{ll} 45 & \\ 6 & rem = 3 \\ 0 & rem = 6 \end{array} \Big\uparrow \quad \Rightarrow \quad (63)_7$$

$$\begin{array}{ll} .75 \times 7 = 5.25 & intpart = 5 \\ 0.25 \times 7 = 1.75 & intpart = 1 \\ 0.75 \times 7 = 5.25 & intpart = 5 \end{array} \quad \Rightarrow \quad (.5151\cdots)_7$$

$$\boxed{(63.5151\cdots)_7}$$

Quiz Time

- Convert $(0.625)_{10}$ to base 2

Quiz Time

- Convert $(0.625)_{10}$ to base 2

$$
\begin{aligned}
.625 \times 2 &= 1.25 && intpart = 1 \\
0.25 \times 2 &= 0.5 && intpart = 0 \\
0.5 \times 2 &= 1.00 && intpart = 1
\end{aligned}
\quad \Longrightarrow \quad (.101)_2
$$

Octal and Hexadecimal Numbers

- Base 8 and base 16 are very useful in computing.
- Hex: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
- Binary to Hex/Octal and vice versa:
  -

$$(673.124)_8 = (\underbrace{110}_{6}\quad \underbrace{111}_{7}\quad \underbrace{011}_{3}\quad .\quad \underbrace{001}_{1}\quad \underbrace{010}_{2}\quad \underbrace{100}_{4})_2$$

$$(306.D)_{16} = (\underbrace{0011}_{3}\quad \underbrace{0000}_{0}\quad \underbrace{0110}_{6}\quad .\quad \underbrace{1101}_{D})_2$$

Binary Arithmetic

Bits		*Sum*	*Carry*	*Difference*	*Borrow*	*Product*
a	b	$a+b$		$a-b$		$a \cdot b$
0	0	0	0	0	0	0
0	1	1	0	1	1	0
1	0	1	0	1	0	0
1	1	0	1	0	0	1

Binary Arithmetic

Binary addition:

$$\begin{array}{rl} 1111 & = \text{carries of } 1 \\ 1111.01 & = (15.25)_{10} \\ 0111.10 & = (\ \ 7.50)_{10} \\ \hline 10110.11 & = (22.75)_{10} \end{array}$$

Binary subtraction:

$$\begin{array}{rl} 1 & = \text{borrows of } 1 \\ 10010.11 & = (18.75)_{10} \\ 01100.10 & = (12.50)_{10} \\ \hline 00110.01 & = (\ \ 6.25)_{10} \end{array}$$

Binary Arithmetic

Binary multiplication:

$$
\begin{array}{r}
11001.1 = (25.5)_{10} \\
110.1 = (\ \ 6.5)_{10} \\
\hline
110011 \\
000000 \\
110011 \\
110011 \\
\hline
10100101.11 = (165.75)_{10}
\end{array}
$$

Binary division:

$$
\begin{array}{r|l}
 & 10110 = \text{quotient} \\
\hline
11001 & 1000100110 \\
 & 11001 \\
\hline
 & 00100101 \\
 & 11001 \\
\hline
 & 0011001 \\
 & 11001 \\
\hline
 & 00000 \quad = \text{remainder}
\end{array}
$$

Complements

- Simplest way to represent negative numbers.
- The most intuitive way to represent a negative number is to use an extra bit for sign — sign-magnitude representation
- But the computation with this representation is little complex, we'll see this later.
- Complements are more handy
- (b-1)'s complement of $(N)_b$: $(b^n - 1) - N$, where n is the number of digits.
- b's complement of $(N)_b$: $b^n - N$, where n is the number of digits

Complements

- Let's say we want to compute $(1234)_{10} - (110)_{10}$
- First we take the 10'complement of 110: $10^4 - 110 = 9890$ (Important!!! Number of digits has to be the same)
- Now add: $1234 + 9890 = 11124$
- $11124 > 10000$: so there will be an end carry in the addition — just discard the carry
- 11124 —> 1124, which is the answer.
- Now, what is the purpose????

Complements — in the binary world

- 2's complement of N: $2^n - N$
- 1's complement of N: $(2^n - 1) - N$
- We have a super easy way to compute these: can you guess why?

Complements in the binary world

- 2's complement of N: $2^n - N$
- 1's complement of N: $(2^n - 1) - N$
- We have a super easy way to compute these: can you guess why?
  - Observe that for any n
    - $(2^n - 1)$ is basically 1111… n times.
    - Now if you subtract N, all the bits of N are flipped.
    - Example: 1111 - 1011 = 0100
  - So, we do not need to do any subtraction really, —just flip the bits.
  - 2's complement = 1's complement + 1

Complements in the binary world

- Compute $M - N$
- Step 1: compute 2's complement of N: $2^n - N = comp(N) + 1$
- Step 2: $M + (2^n - N) = 2^n + (M - N)$
- Now if $M \geq N$: the sum will be $> 2^n$ — so there will be a carry. Just discard it and output the result.
- If $M < N$ the result is $2^n + (N - M) < 2^n$. This is basically the 2's complement of $(N - M)$. No carry will be produced. The output will be 2's complement of the result, with a negative sign in the front.

Complements in the binary world

- Let X = 1010100, Y = 1000011; compute X-Y and Y - X.

Complements in the binary world

- Let X = 1010100, Y = 1000011; compute X-Y and Y - X.

$$
\begin{aligned}
X &= 1010100 \\
\text{2's complement of } Y &= +\,\underline{0111101} \\
\text{Sum} &= 10010001 \\
\text{Discard end carry } 2^7 &= -\underline{10000000} \\
\text{Answer: } X - Y &= 0010001
\end{aligned}
$$

$$
\begin{aligned}
Y &= 1000011 \\
\text{2's complement of } X &= +\,\underline{0101100} \\
\text{Sum} &= 1101111
\end{aligned}
$$

There is no end carry.

Answer: $Y - X = -(\text{2's complement of } 1101111) = -0010001$

- Important!!! We are doing unsigned subtraction!!
- Food of thought: we can do the same with 1's complement too!!, then why 2's complement???

Complements in the binary world

- We can do the same with 1's complement too!!, then why 2's complement???
-

$+N$	Positive Integers (all systems)	$-N$	Negative Integers		
			Sign and Magnitude	2's Complement N^*	1's Complement $\bar{N}$
+0	0000	−0	1000	——	1111
+1	0001	−1	1001	1111	1110
+2	0010	−2	1010	1110	1101
+3	0011	−3	1011	1101	1100
+4	0100	−4	1100	1100	1011
+5	0101	−5	1101	1011	1010
+6	0110	−6	1110	1010	1001
+7	0111	−7	1111	1001	1000
		−8	——	1000	——

- For signed magnitude, we use 4 bits to represent [-7,7] with the 4th bit being the sign bit

Complements in the binary world

- We can do the same with 1's complement too!!, then why 2's complement???
-

$+N$	Positive Integers (all systems)	$-N$	Negative Integers		
			Sign and Magnitude	2's Complement N^*	1's Complement $\bar{N}$
+0	0000	−0	1000	——	1111
+1	0001	−1	1001	1111	1110
+2	0010	−2	1010	1110	1101
+3	0011	−3	1011	1101	1100
+4	0100	−4	1100	1100	1011
+5	0101	−5	1101	1011	1010
+6	0110	−6	1110	1010	1001
+7	0111	−7	1111	1001	1000
		−8	——	1000	——

- For 2's complement, we still use 4 bits to represent [-7,7]; but here the encoding is different.
- 1's complement has a negative 0

Complements in the binary world

System	Range
Unsigned	$[0, 2^N - 1]$
Sign/Magnitude	$[-2^{N-1} + 1, 2^{N-1} - 1]$
Two's Complement	$[-2^{N-1}, 2^{N-1} - 1]$

Thank you

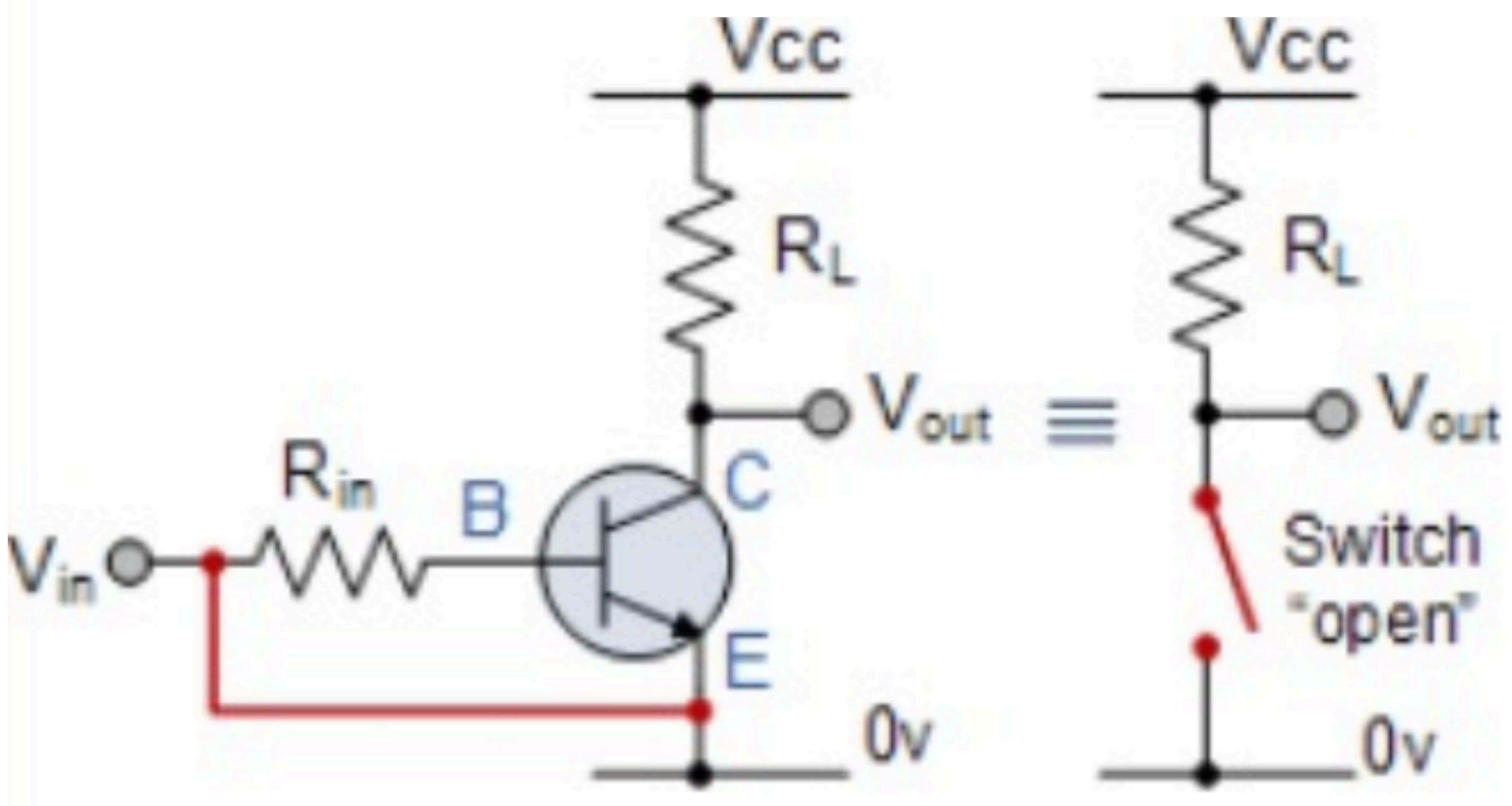
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

Switching Algebra

Switches



Algebra of Switches

- **Warning!!! Too informal**: An algebraic system consists of a set and some well-defined operators on the elements of the set and possibly some relations defined over the elements.
- We will be studying one of the simplest system defined over the set $\{0, 1\}$ — but ironically this set is going to complicate the rest of your life. :P (just joking)
- **Set**: $\{0,1\}$
- **Operators**:
  - AND, OR (Binary)
  - NOT (Unary)

$$
\begin{aligned}
0 + 0 &= 0, \\
0 + 1 &= 1, \\
1 + 0 &= 1, \\
1 + 1 &= 1.
\end{aligned}
\qquad
\begin{aligned}
0 \cdot 0 &= 0, \\
0 \cdot 1 &= 0, \\
1 \cdot 0 &= 0, \\
1 \cdot 1 &= 1.
\end{aligned}
\qquad
\begin{aligned}
0' &= 1, \\
1' &= 0.
\end{aligned}
$$

Algebra of Switches: Basic Properties

- **Idempotency**: $x + x = x$

  $x \,.\, x = x$

How do we prove this property?

Perfect induction*: proving a theorem by verifying every combination of values that the variables may assume*

Proof of $\boldsymbol{x + x = x}$: $1 + 1 = 1$ and $0 + 0 = 0$

- If x is a switching variable, then: $x + 1 = 1$

  $x \,.\, 0 = 0$

  $x + 0 = x$

  $x \,.\, 1 = x$

- **Commutativity**: $x + y = y + x$

  $x \,.\, y = y \,.\, x$

Algebra of Switches: Basic Properties

- **Associativity**: $(x + y) + z = x + (y + z)$
  $(x \,.\, y) \,.\, z = x \,.\, (y \,.\, z)$

- **Complementation**: $x + x' = 1$
  $x \,.\, x' = 0$

- **Distributivity**: $x \,.\, (y + z) = x \,.\, y + x \,.\, z$
  $x + y \,.\, z = (x + y) \,.\, (x + z)$

Proofs are easy, you can write down the entire truth table

x	y	z	xy	xz	$y+z$	$x(y+z)$	$xy+xz$
0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	0	0	1	0	0
1	0	0	0	0	0	0	0
1	0	1	0	1	1	1	1
1	1	0	1	0	1	1	1
1	1	1	1	1	1	1	1

Algebra of Switches: Principle of Duality

- **Observe**: Preceding properties grouped in pairs
- One statement can be obtained from the other by interchanging operations OR and AND and constants 0 and 1
- The two statements are said to be dual of each other
- **Implication**: necessary to prove only one of each pair of statements

Algebra of Switches: Some Basic Laws

- **Switching expression**: combination of finite number of switching variables and constants via switching operations (AND, OR, NOT)
  - Any constant or switching variable is a switching expression
  - If T_1 and T_2 are switching expressions, so are T_1', T_2', T_1+T_2 and T_1T_2
  - No other combination of constants and variables is a switching expression

- **Absorption law**: $x + xy = x$

  $x(x + y) = x$

Proof: $x + xy = x1 + xy$ [basic property]

$= x(1 + y)$ [distributivity]

$= x1$ [commutativity and basic property]

(Note we write in the law y + 1 = y, so we apply commutativity)

$= x$ [basic property]

Algebra of Switches: Some Basic Laws

- **Switching expression**: combination of finite number of switching variables and constants via switching operations (AND, OR, NOT)
  - Any constant or switching variable is a switching expression
  - If T_1 and T_2 are switching expressions, so are T_1', T_2', T_1+T_2 and T_1T_2
  - No other combination of constants and variables is a switching expression

- **Absorption law**: $x + xy = x$

  $x(x + y) = x$

Proof: $x + xy = x1 + xy$ [basic property]

$= x(1 + y)$ [distributivity]

$= x1$ [commutativity and basic property]

(Note we write in the law y + 1 = y, so we apply commutativity)

$= x$ [basic property]

Simplification of Switching Expressions

- **Law 2:** $x + x'y = x + y$, $x(x' + y) = xy$
- **Law 3**: $xy + x'z + yz = xy + x'z$, $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$. **(Consensus Theorem)**
- **Law 4**: $(x')' = x$. **(Involution)**
- **Law 5**: $(x+y)' = x'y'$, $(xy)' = x' + y'$ **(De-Morgan's Theorem)**

Food of thought: Can we extend it for n variables???

x	y	x'	y'	$x+y$	$(x+y)'$	$x'y'$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

Try the proofs!!!

Simplification of Switching Expressions

Example: Simplify $T(x,y,z) = (x + y)[x'(y' + z')]' + x'y' + x'z'$

Simplification of Switching Expressions

Example: Simplify $T(x,y,z) = (x + y)[x'(y' + z')]' + x'y' + x'z'$

$$
\begin{aligned}
(x + y)[x'(y' + z')]' + x'y' + x'z' &= (x + y)(x + yz) + x'y' + x'z' \\
&= (x + xyz + yx + yz) + x'y' + x'z' \\
&= x + yz + x'y' + x'z' \\
&= x + yz + y' + z' \\
&= x + z + y' + z' \\
&= x + y' + 1 \\
&= 1
\end{aligned}
$$

Thus, $T(x,y,z) = 1$, independently of the values of the variables

Simplification of Switching Expressions

Example: Prove $xy + x'y' + yz = xy + x'y' + x'z$

Simplification of Switching Expressions

Example: Prove $xy + x'y' + yz = xy + x'y' + x'z$

- From consensus theorem, $x'z$ can be added to LHS
- Consensus theorem can be applied again to first, third and fourth terms in $xy + x'y' + yz + x'z$ to eliminate yz and reduce it to RHS

Simplification of Switching Expressions

Example: Prove $xy + x'y' + yz = xy + x'y' + x'z$

- From consensus theorem, $x'z$ can be added to LHS
- Consensus theorem can be applied again to first, third and fourth terms in $xy + x'y' + yz + x'z$ to eliminate yz and reduce it to RHS

- LHS : $xy + x'y' + yz = xy + (x'y' + yz) = xy + (y'x' + (y')'z + x'z) = xy + y'x' + yz + x'z = (xy + x'z + yz) + x'y' = xy + x'z + x'y' = RHS$

Switching Functions

- **Switching function $f(x_1, x_2, \ldots, x_n)$**: values assumed by an expression for all combinations of variables $x_1, x_2, \ldots, x_n$
- **Complement function**: $f'(x_1, x_2, \ldots, x_n)$ assumes value 0 (1) whenever $f(x_1, x_2, \ldots, x_n)$ assumes value 1 (0)
- **Logical sum of two functions**: $f(x_1, x_2, \ldots, x_n) + g(x_1, x_2, \ldots, x_n) = 1$ for every combination in which either f or g or both equal 1
- **Logical product of two functions**: $f(x_1, x_2, \ldots, x_n) \cdot g(x_1, x_2, \ldots, x_n) = 1$ for every combination for which both f and g equal 1

x	y	z	f	g	f'	$f+g$	fg
0	0	0	1	0	0	1	0
0	0	1	0	1	1	1	0
0	1	0	1	0	0	1	0
0	1	1	1	1	0	1	1
1	0	0	0	1	1	1	0
1	0	1	0	0	1	0	0
1	1	0	1	1	0	1	1
1	1	1	1	0	0	1	0

Canonical Forms

- Truth tables are one of the most standard way of representing switching functions.
- But a functions with n variables require 2^n rows each of size $(n+1)$ bits. — quite some overhead.
- **Let's consider the problem of deriving a compact expression of a function from a given truth table**

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

$$f = x'y'z' + x'yz' + x'yz + xyz' + xyz$$

- Find the sum (OR) of all terms for which the function evaluates to 1.
  - In this case, for (0, 2, 3, 6, 7).
- Each term is a product of the variables on which the function depends
- Variable x_i appears in uncomplemented (complemented) form in the product if has value 1 (0) in the combination

Canonical Forms: Sum of Products

- **Minterm**: a product term that contains each of the *n* variables as factors in either complemented or uncomplemented form
    - It assumes value 1 for exactly one combination of variables
    - In the following table 000, 010, 011, 110, 111 are minterms, which are usually represented with their decimal equivalent (0,2,3,6,7). In terms of variables *x'y'z', x'yz', x'yz, xyz', xyz*

- **Canonical sum-of-products**: sum of all minterms derived from combinations for which function is 1
    - Also called **disjunctive normal expression**

- **Compact representation**: $\sum(0,2,3,6,7)$

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Canonical Forms: Product of Sums

- **Maxterm**: a sum term that contains each of the *n* variables in either complemented or uncomplemented form
  - It assumes value 0 for exactly one combination of variables
  - Variable x_i appears in uncomplemented (complemented) form in the sum if it has value 0 (1) in the combination
- **Canonical product-of-sums**: product of all maxterms derived from combinations for which function is 0
  - Also called **conjunctive normal expression**

- **Compact representation**: $\prod(1,4,5)$

$$f = (x + y + z')(x' + y + z)(x' + y + z')$$

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Derivation of SOP Forms

1. Examine each term: if it is a minterm, retain it; continue to next term

2. In each product which is not a minterm: check the variables that do not occur; for each x_i that does not occur, multiply the product by $(x_i + x_i')$

3. Multiply out all products and eliminate redundant terms

Example: $T(x,y,z) = x'y + z' + xyz$

$$= x'y(z + z') + (x + x')(y + y')z' + xyz$$
$$= x'yz + x'yz' + xyz' + xy'z' + x'yz' + x'y'z' + xyz$$
$$= x'yz + x'yz' + xyz' + xy'z' + x'y'z' + xyz$$

Canonical product-of-sums obtained in a dual manner

Example:

$T = x'(y' + z)$

$$= (x' + yy' + zz')(y' + z + xx')$$
$$= (x' + y + z)(x' + y + z')(x' + y' + z)(x' + y' + z')(x + y' + z)(x' + y' + z)$$
$$= (x' + y + z)(x' + y + z')(x' + y' + z)(x' + y' + z')(x + y' + z)$$

Transforming Canonical Forms

- **Example**: Find the canonical product-of-sums for

$$T(x,y,z) = x'y'z' + x'y'z + x'yz + xyz + xy'z + xy'z'$$

$$T = (T')' = [(x'y'z' + x'y'z + x'yz + xyz + xy'z + xy'z')']'$$

  Complement T' consists of minterms not contained in T. Thus,

$$T = [x'yz' + xyz']' = (x + y' + z)(x' + y' + z)$$

- Canonical forms are unique

- **Two switching functions are equivalent if and only if their corresponding canonical forms are identical**

Introducing Exclusive-OR (XOR)

Exclusive-OR: modulo-2 addition, i.e., $A \oplus B = 1$ if either A or B is 1, but not both.

Commutativity: $A \oplus B = B \oplus A$

Associativity: $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

Distributivity: $(AB) \oplus (AC) = A(B \oplus C)$

If $A \oplus B = C$, then

$$A \oplus C = B$$
$$B \oplus C = A$$
$$A \oplus B \oplus C = 0$$

Exclusive-OR of an even (odd) number of elements, whose value is 1, is 0 (1)

Functional Completeness

- A set of operations is **functionally complete** (or **universal**) if and only if every switching function can be expressed by operations from this set

- Every switching function can be expressed in canonical form consisting of a finite number of switching variables, constants and operations OR, AND, NOT

- **Example**: Set {+, ., ’}
- **Example**: Set {+, ’} since using De Morgan’s theorem, $x \,.\, y = (x' + y')'$. Thus, + and ’ can replace the . in any switching function
- **Example**: Set {., ’} for similar reasons
- **Example**: NAND since NAND(x,x) $= x'$ and NAND[NAND(x,y),NAND(x,y)] = xy
- **Example**: NOR since NOR(x,x) $= x'$ and NOR[NOR(x,y),NOR(x,y)] $= x + y$

What Can be done with Switching Algebra?

Isomorphism: Two algebraic systems are isomorphic if

- For every operation in one system, there exists a corresponding operation in the second system
- To each element x_i in one system, there corresponds a unique element y_i in the other system, and vice versa
- If each operation and element in every postulate of one system is replaced by the corresponding operation and element in the other system, then the resulting postulate is valid in the second system

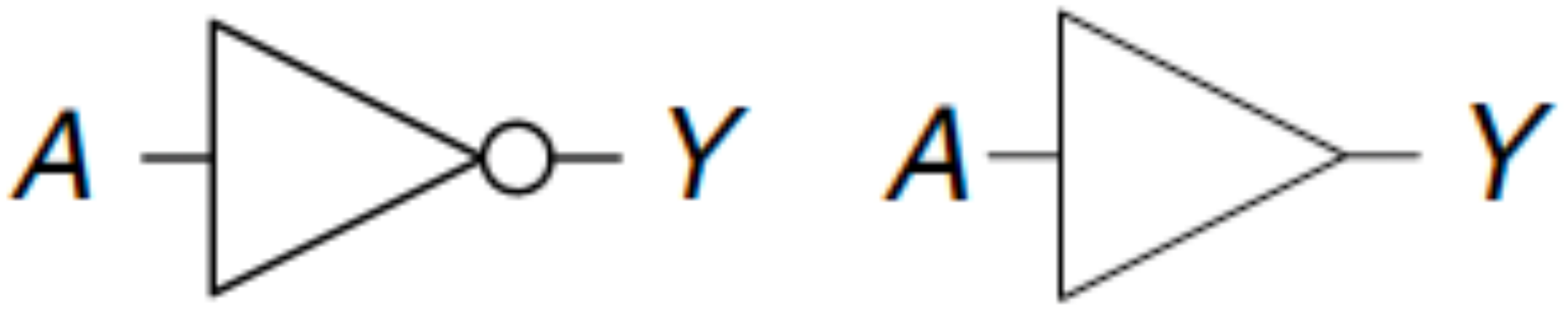
Thus, two algebraic systems are isomorphic if and only if they are identical except the labels and symbols used to represent the operations and elements.

- Switching algebra is isomorphic to proposition calculus — something which encodes declarative statements with values "True" or "False", but never both.
  - "If it rains, then we play"

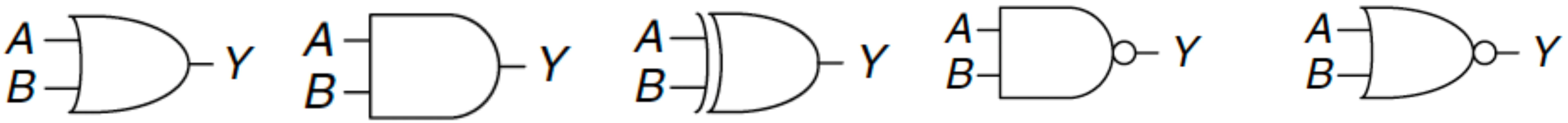
- **Most important**!!!Switching algebra is isomorphic to the **network of logic gates**, something which we build with transistors.

At The Gates

NOT BUF



OR AND XOR NAND NOR



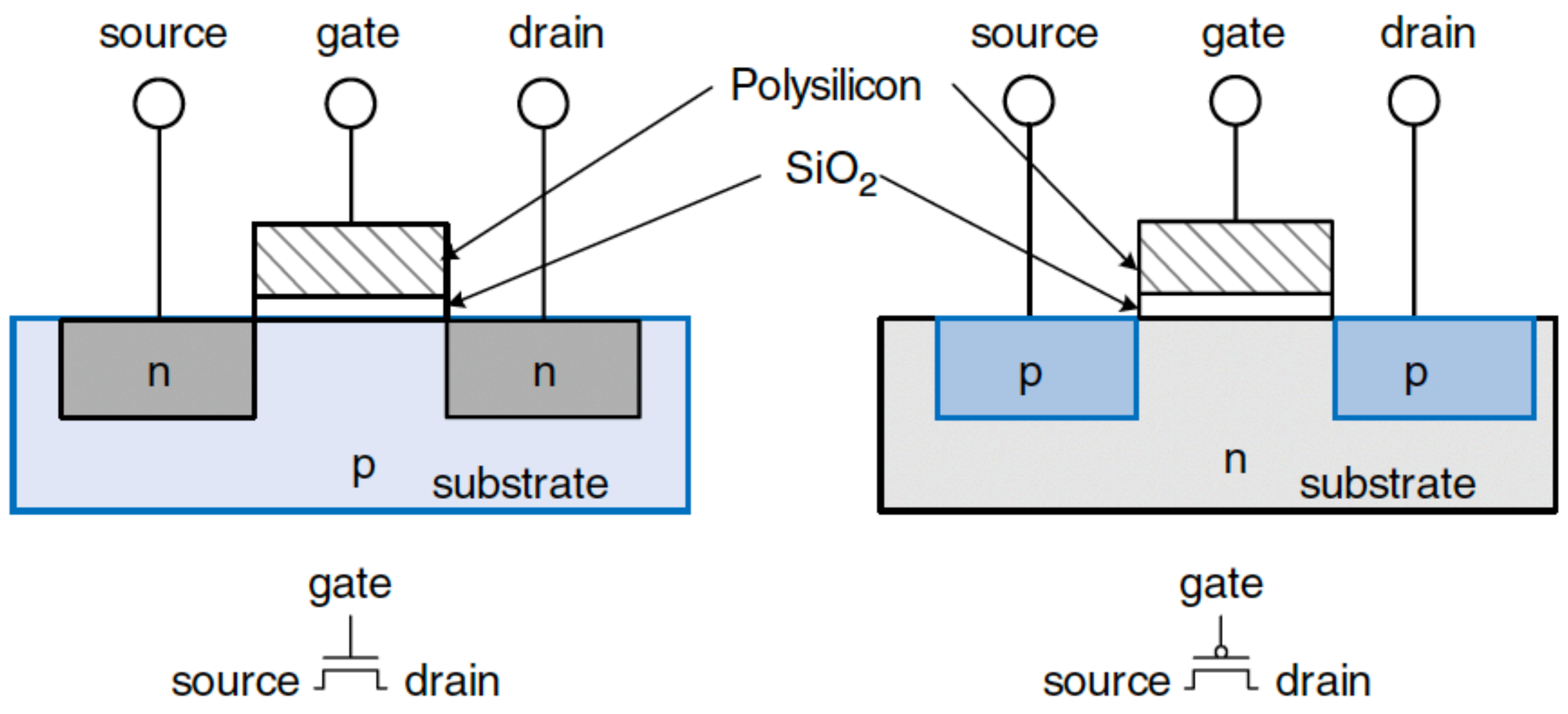
NAND: $Y = \overline{AB}$

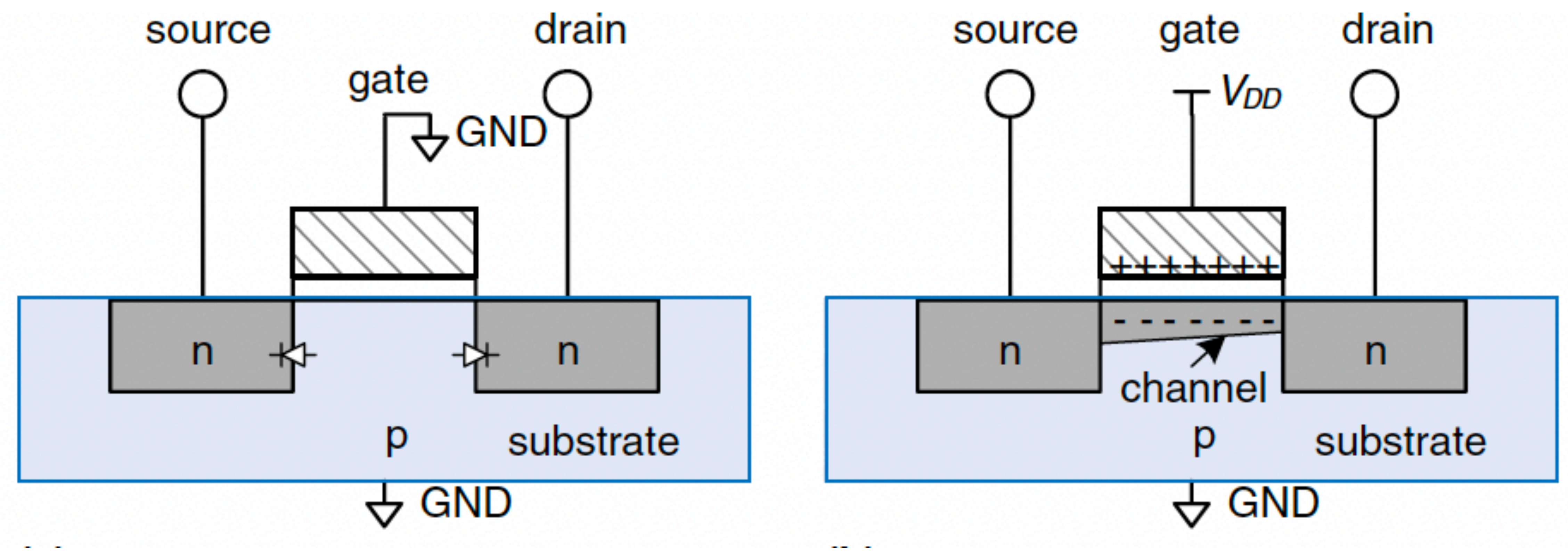
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

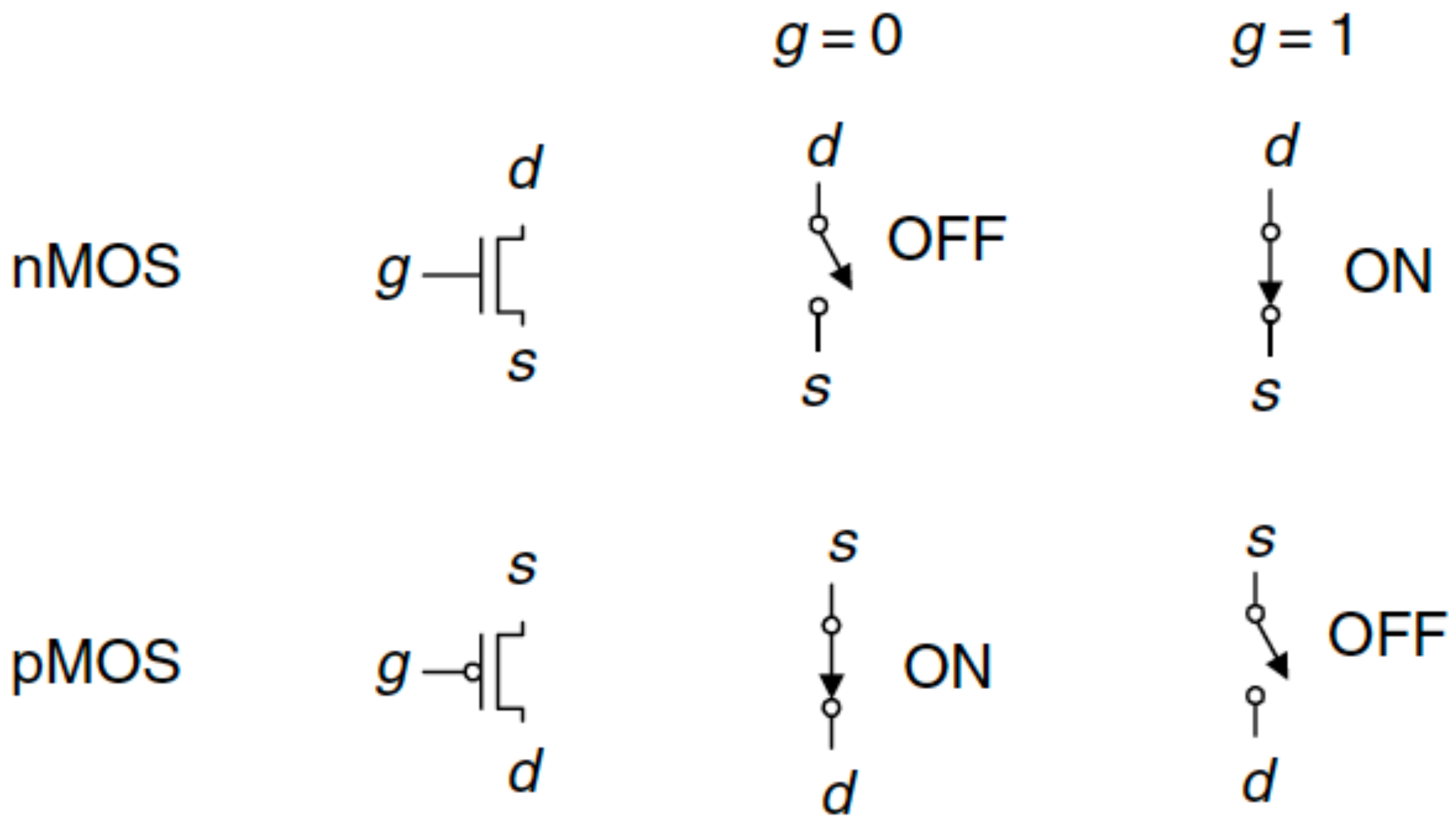
NOR: $Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

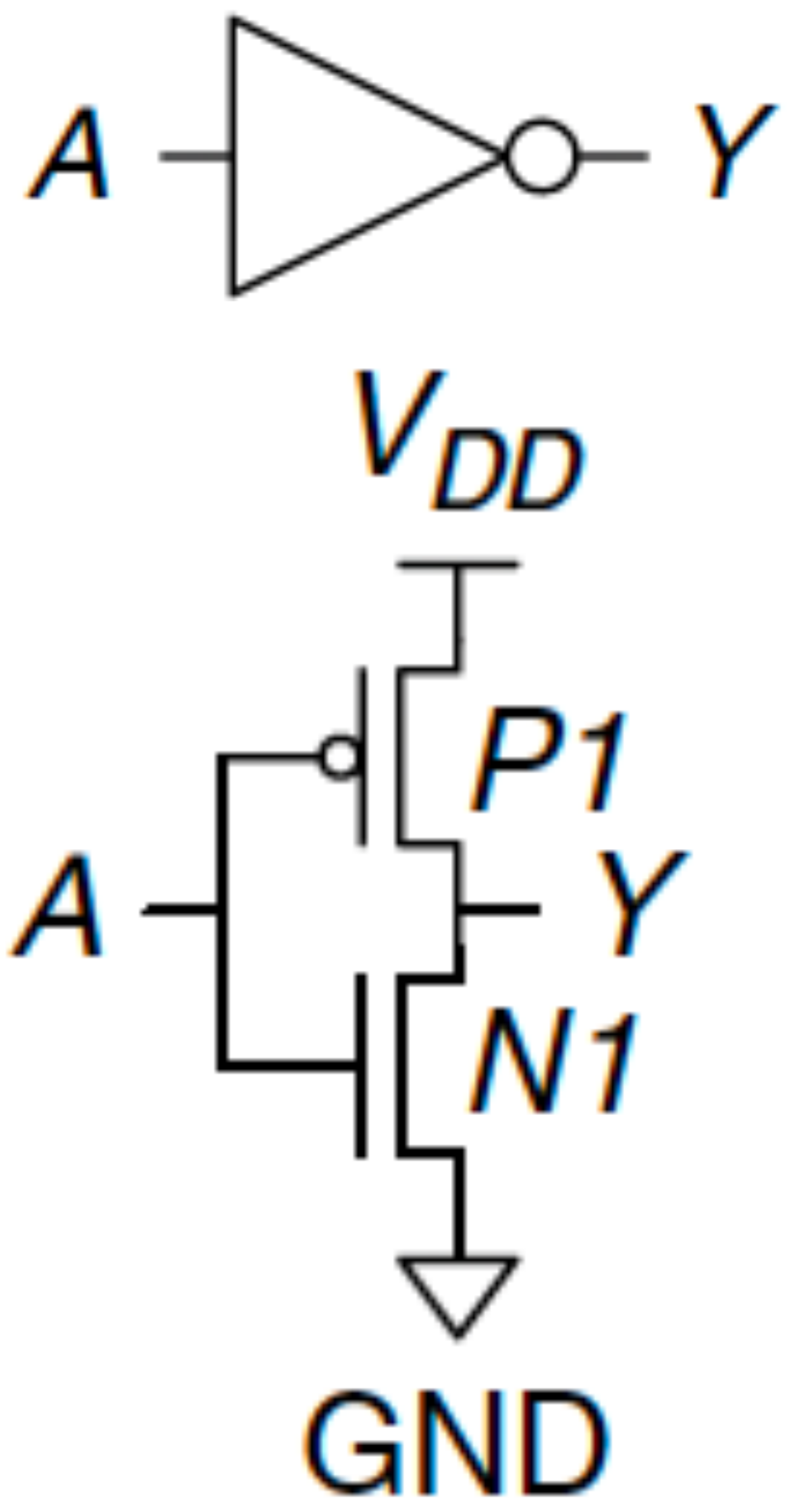
One Level Below

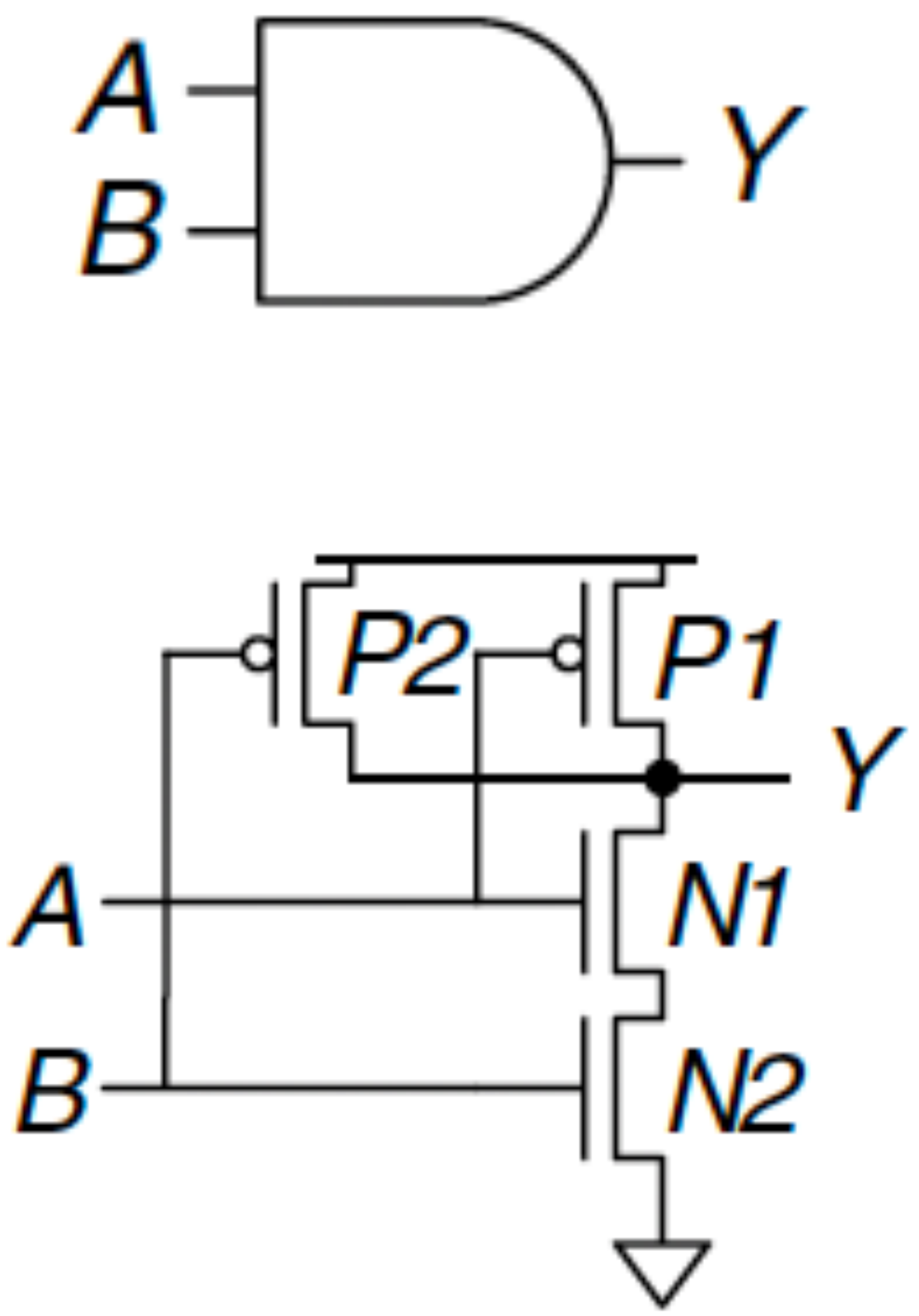


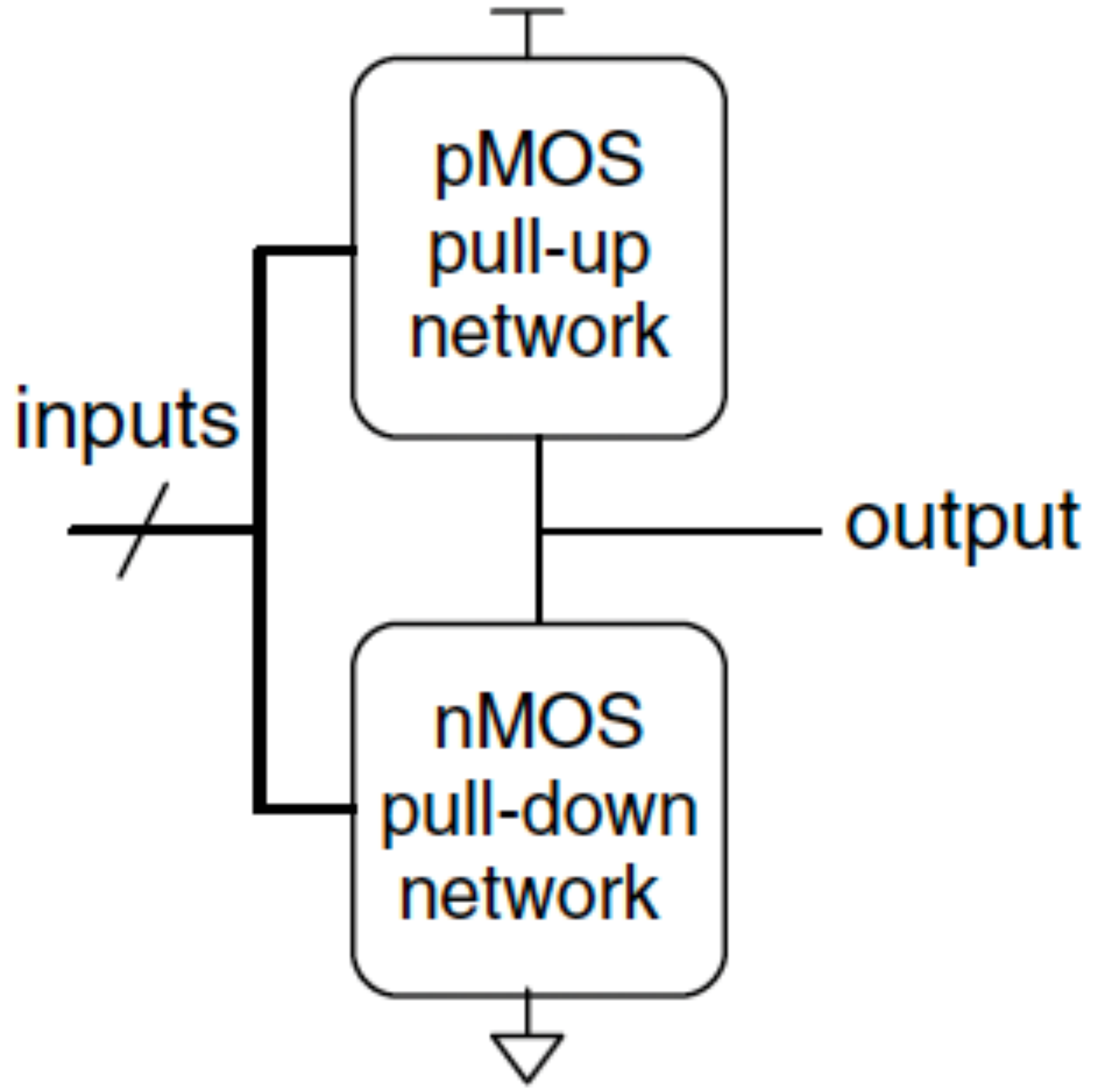




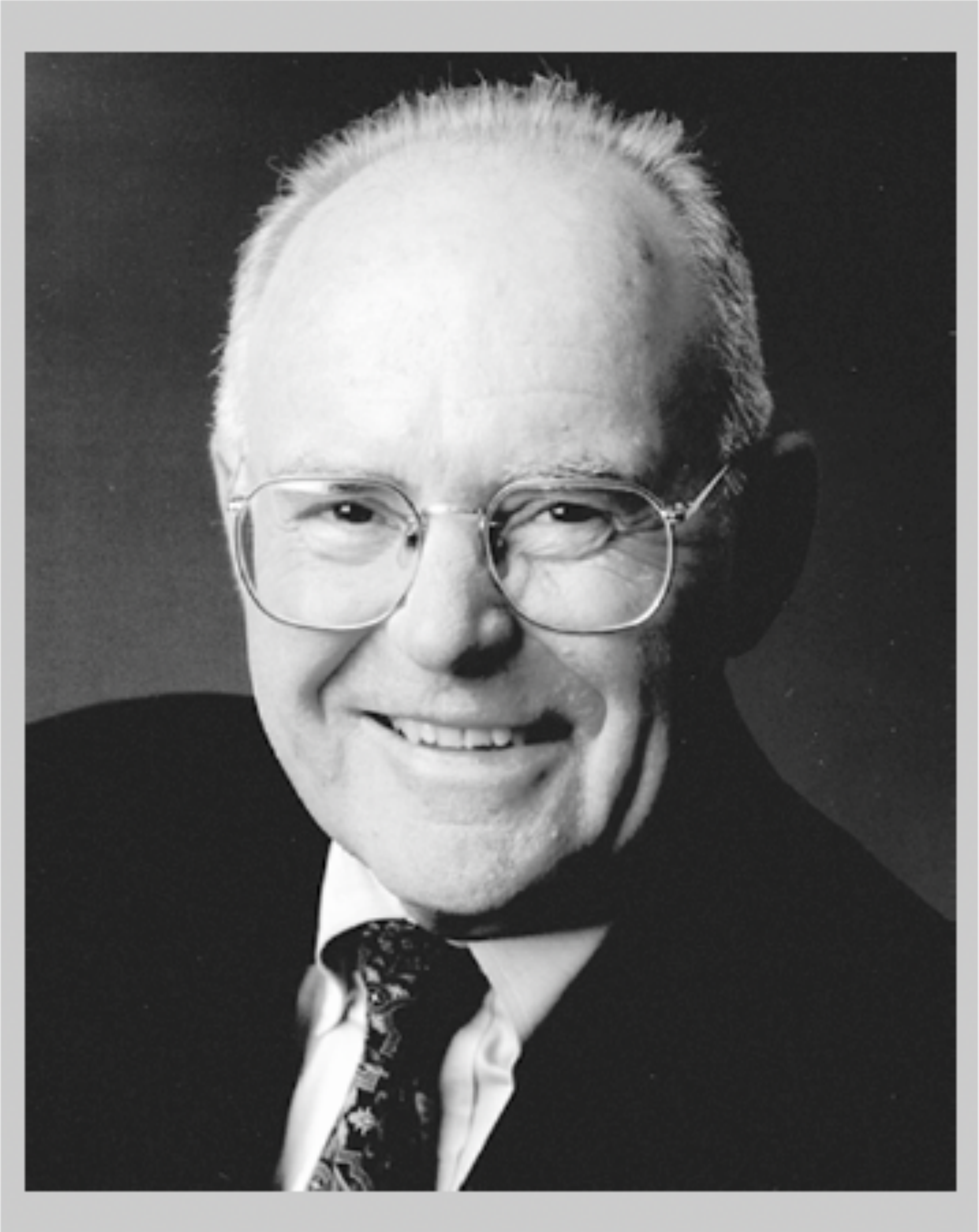
CMOS Level View







A Bit of GK



Thank you

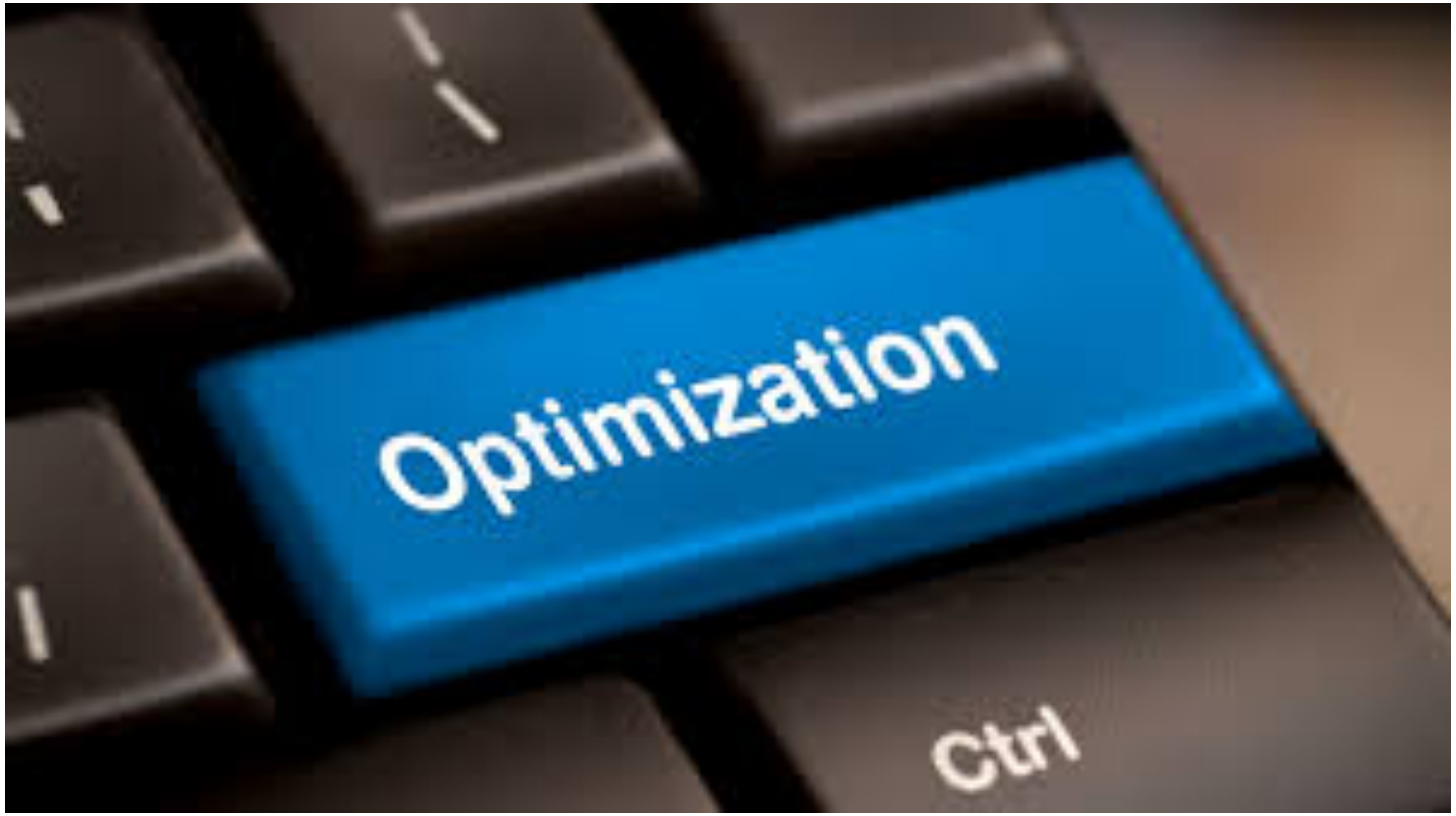
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

Logic Minimization

Life of an Engineer

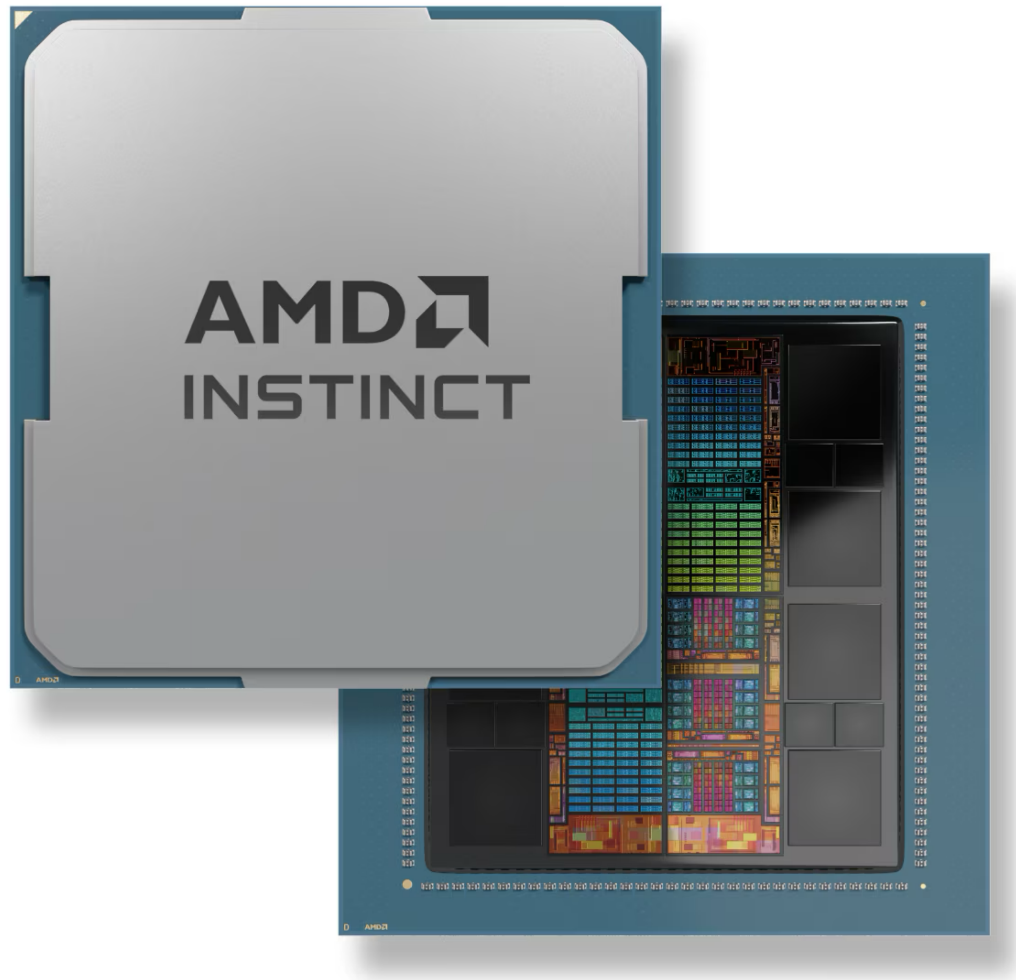


Logic Minimization: Why?

- Consider a switching expression. **How many gates do you need to implement this?** Consider each gate is 2-input, 1 output except the NOTs — 5 OR, 12 ANDs, 3 NOTs
$$f(x, y, z) = x'yz' + x'y'z' + xy'z' + x'yz + xyz + xy'z$$
- Now consider the following expression:' $x'z' + y'z' + yz + xz$
- Observe that both implements the same logic function!!! Now you need 4 ORs, 4 ANDs, and 3 NOTs.
- Can you do better?? — Yes $f(x, y, z) = x'z' + xy' + yz$
- Turns out that there can be more such expressions.
- Lower gate count => Lower transistor count => Lower area (and perhaps less power, and time)…
- So, now we have an engineering problem in hand — **how to minimize the switching expressions???**

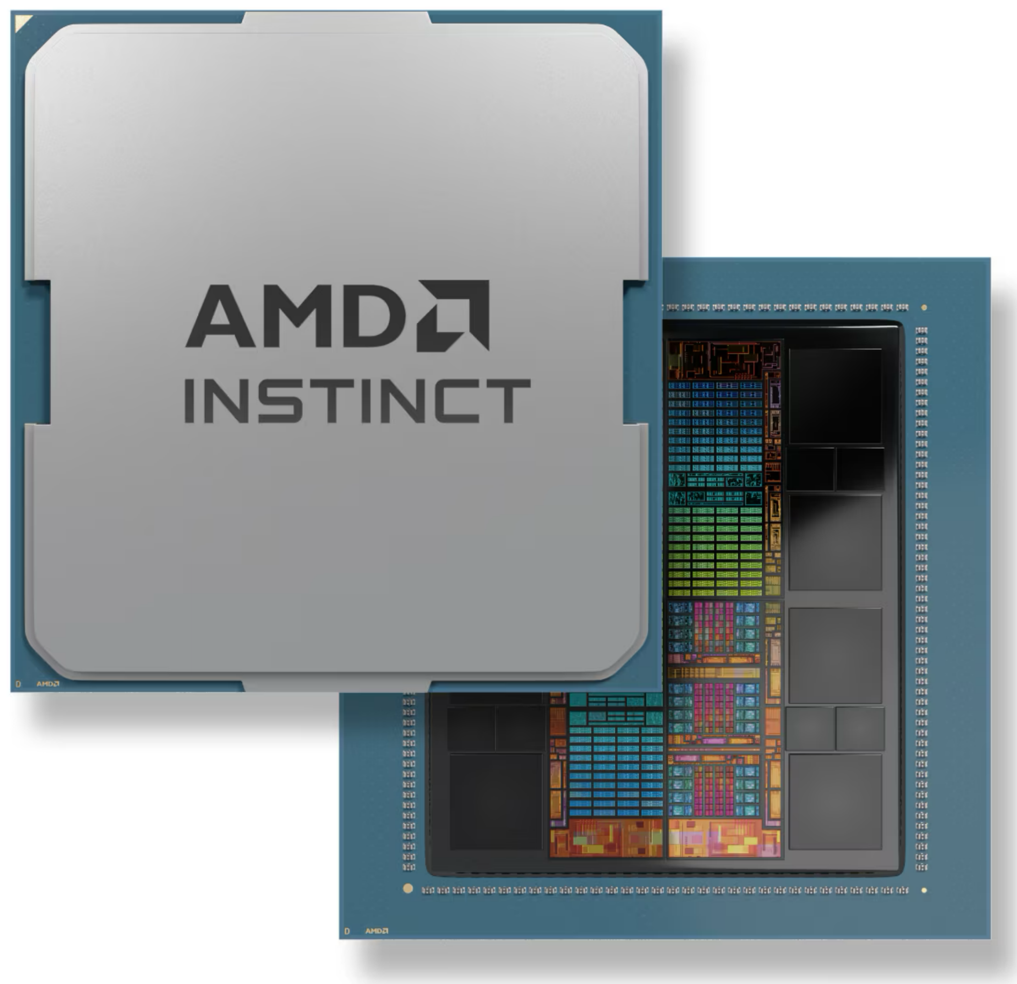
Bigger Picture

- Modern circuits contains billions of gates — e.g. AMD Instinct is a GPU processor containing 146,000,000,000 transistors; so a few billions of gates (if not trillions)…
- How do people minimized their gate network…Fortunately we have tools for that.
- Today we will be studying some of the fundamental techniques behind these tools.



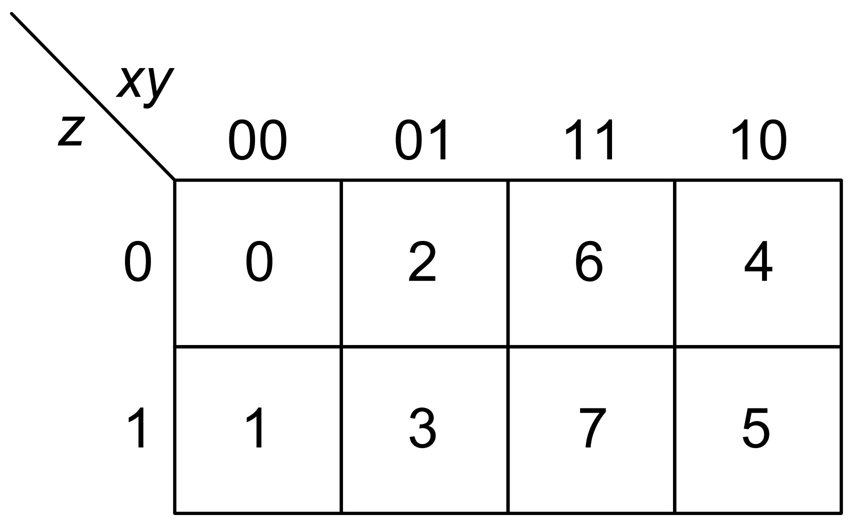
Bigger Picture

- Modern circuits contains billions of gates — e.g. AMD Instinct is a GPU processor containing 146,000,000,000 transistors; so a few billions of gates (if not trillions)…
- How do people minimized their gate network…Fortunately we have tools for that.
- Today we will be studying some of the fundamental techniques behind these tools.
  - Of course, a very very rudimentary intro

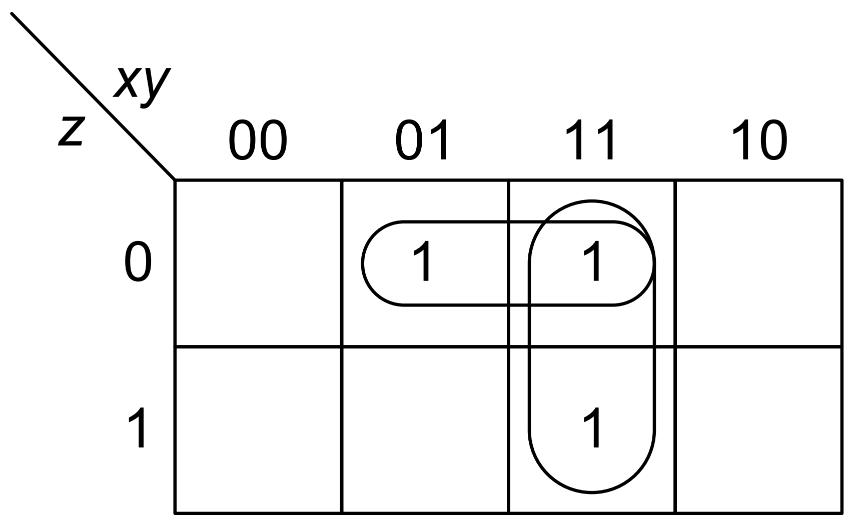


The Map Method

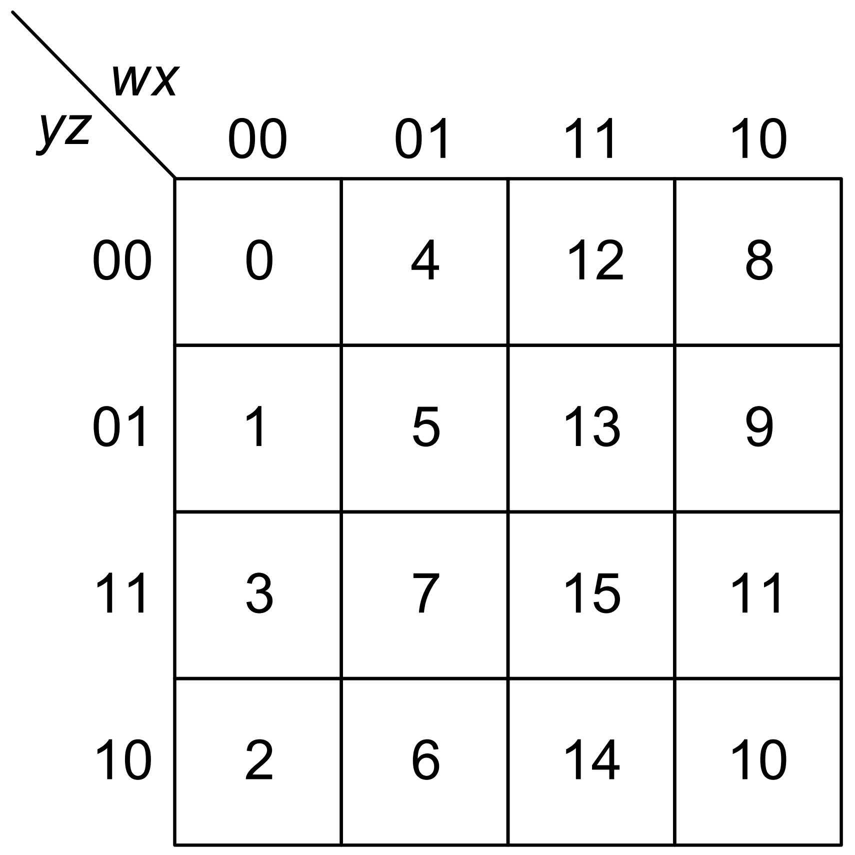
- **Karnaugh map**: modified form of truth table
- Combine terms using the **Aa + Aa’ = A (combining theorem)**



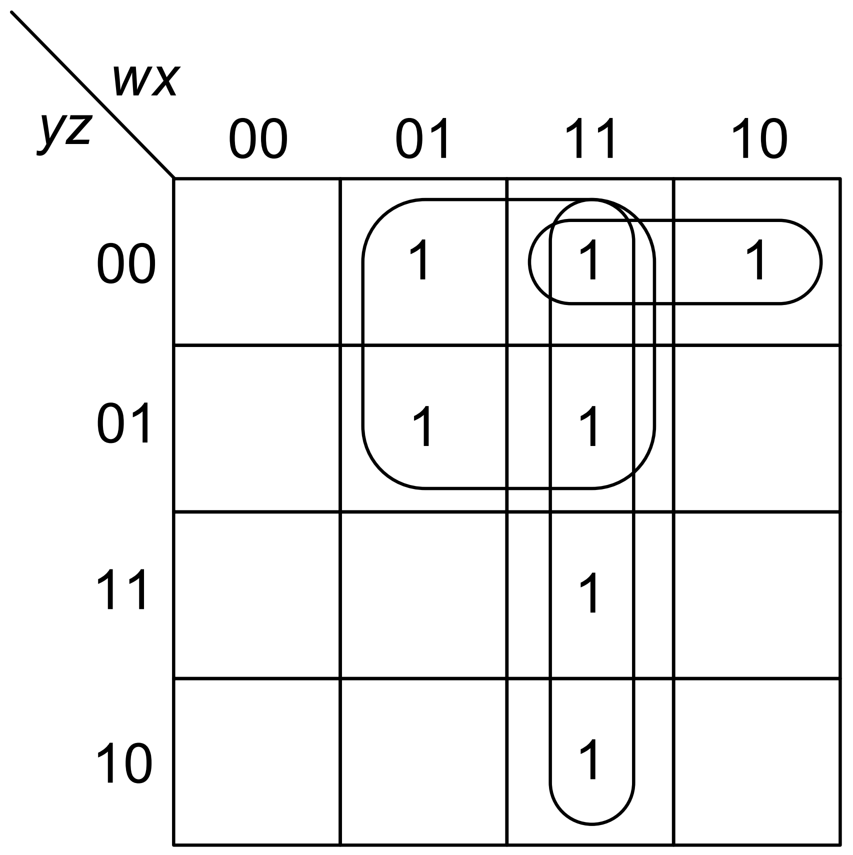
(a) Location of minterms in a three-variable map.



(b) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.



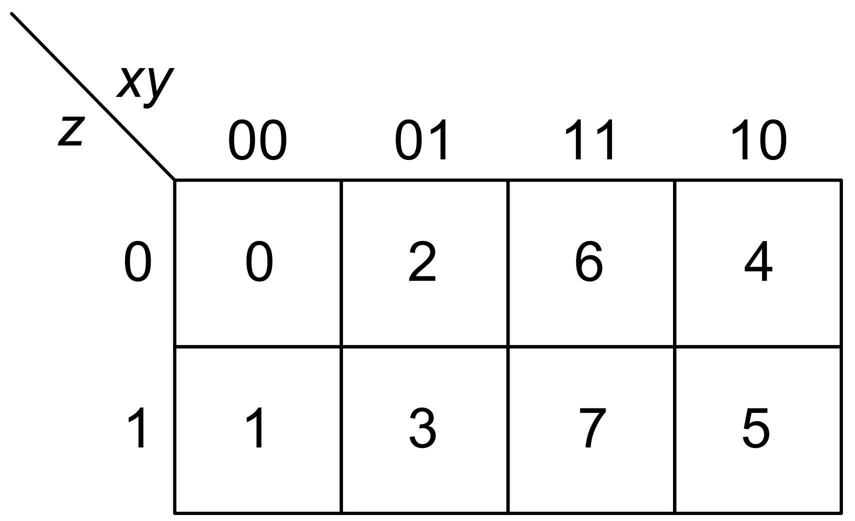
(c) Location of minterms in a four-variable map.



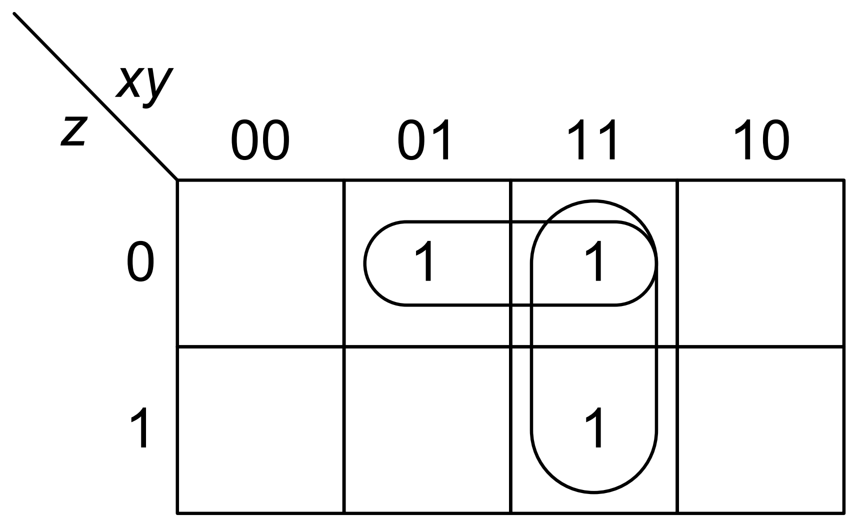
(d) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

The Map Method

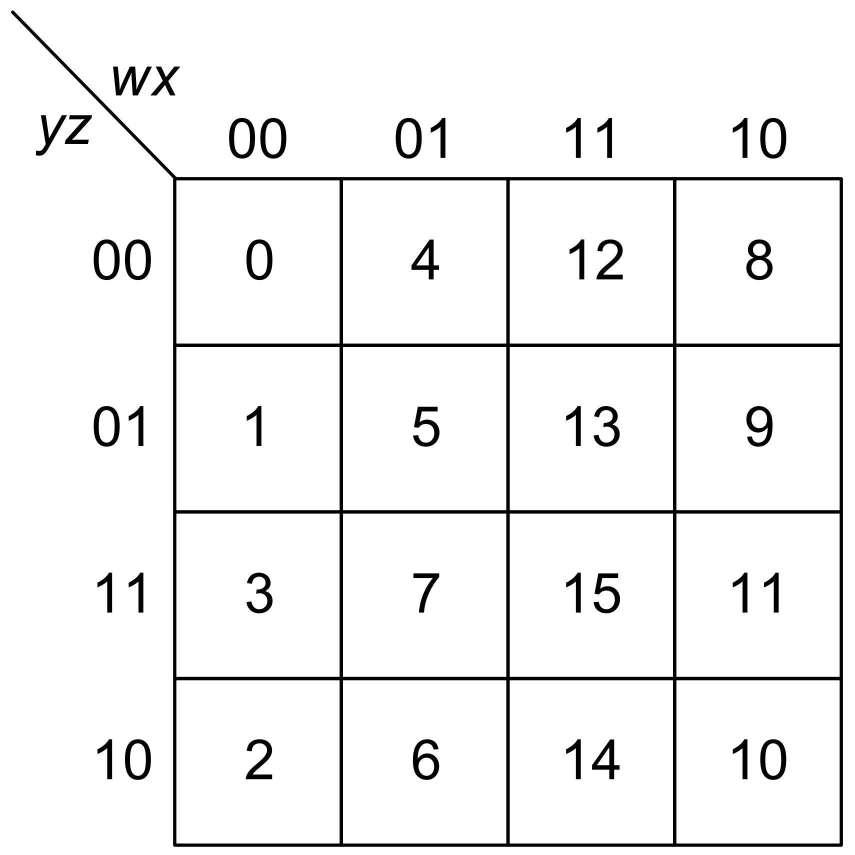
- **Karnaugh map**: modified form of truth table
- Combine terms using the **Aa + Aa' = A (combining theorem)**
- **Cube**:
  - Collection of $\mathbf{2^{m}}$ cells, each adjacent to ***m*** cells of the collection
  - The cube is said to **cover** the cells it is involved with
  - Expressed by a product of **n-m literals** for a function containing **n variables**
  - **m literals** not in the product said to be eliminated



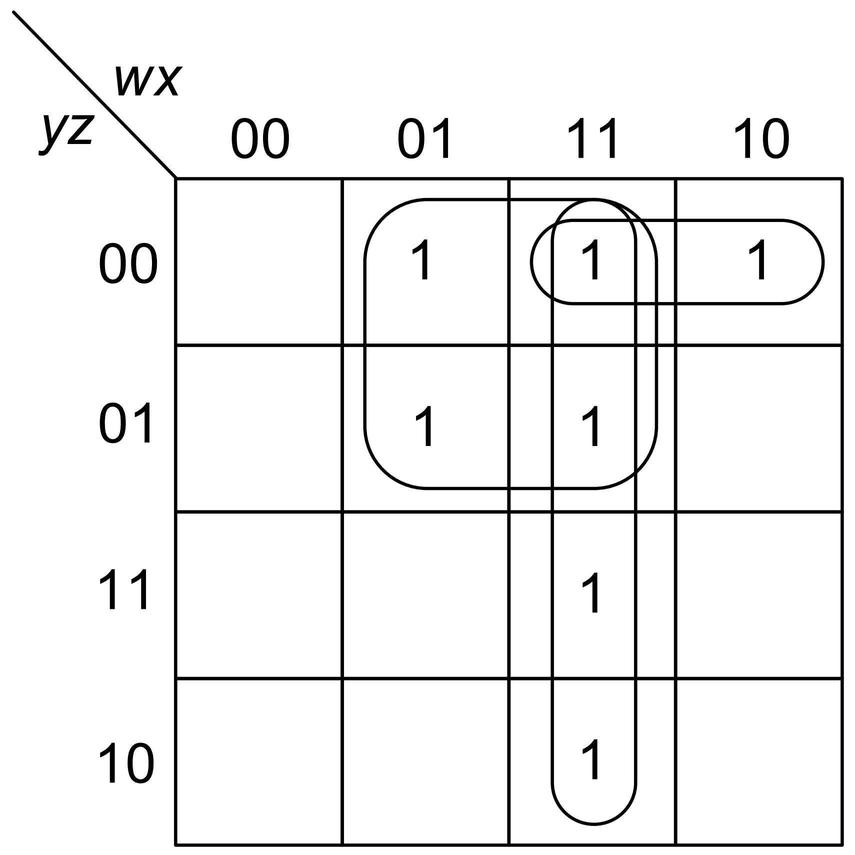
(*a*) Location of minterms in a three-variable map.



(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.



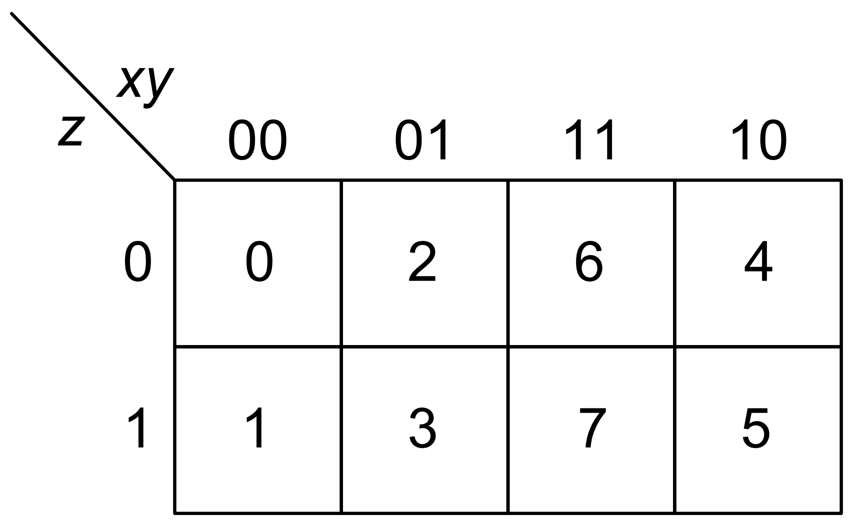
(*c*) Location of minterms in a four-variable map.



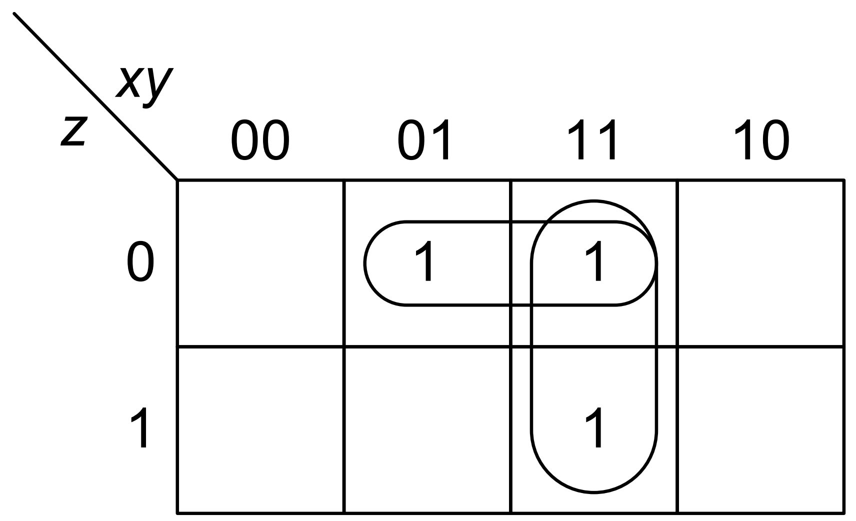
(*d*) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

The Map Method

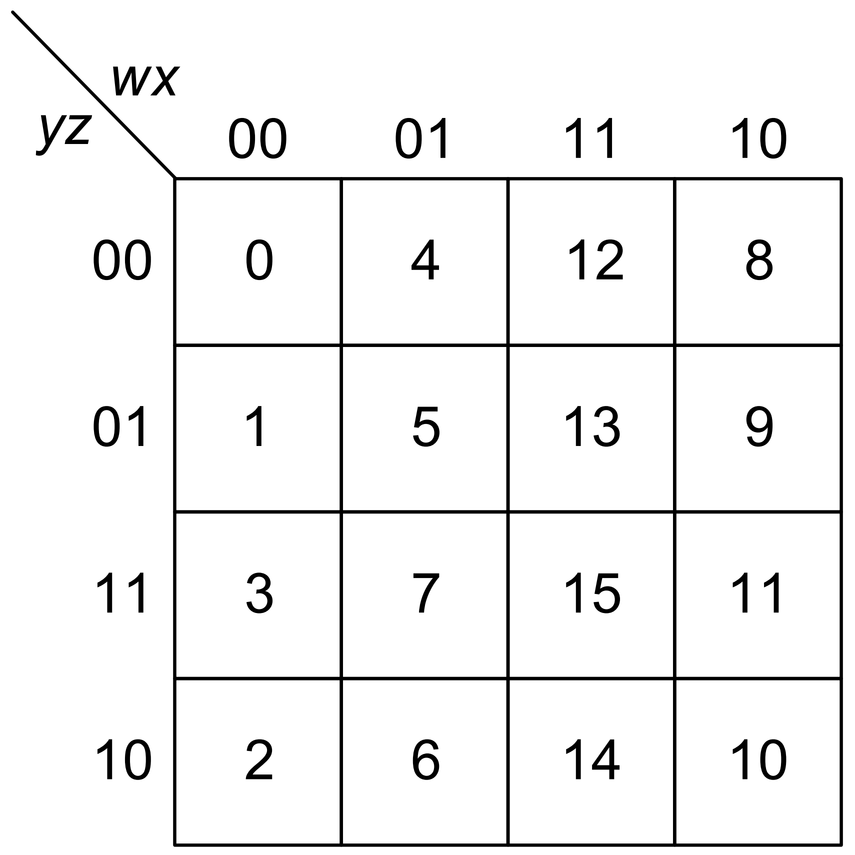
- **Karnaugh map**: modified form of truth table
- Combine terms using the **Aa + Aa’ = A (combining theorem)**
- **<u>Cube</u>**:
  - Collection of **2^m** cells, each adjacent to ***m*** cells of the collection
  - The cube is said to **cover** the cells it is involved with
  - Expressed by a product of **n-m literals** for a function containing **n variables**
  - **m literals** not in the product said to be eliminated
  - **More Clarification**:
    - Consider the squares 2 and 6 in Fig (a)
    - The minterms are z’x’y and z’xy
    - Now apply the **combining theorem.**
    - Literal x and x’ are eliminated.
    - The result is a 2-cube.



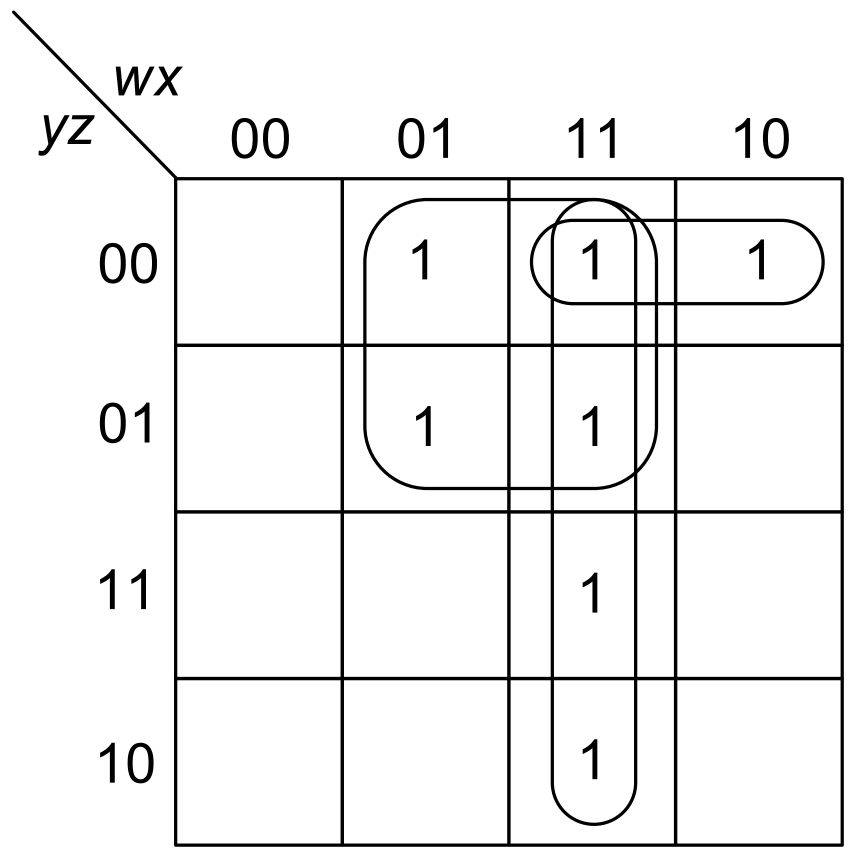
(*a*) Location of minterms in a three-variable map.



(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.

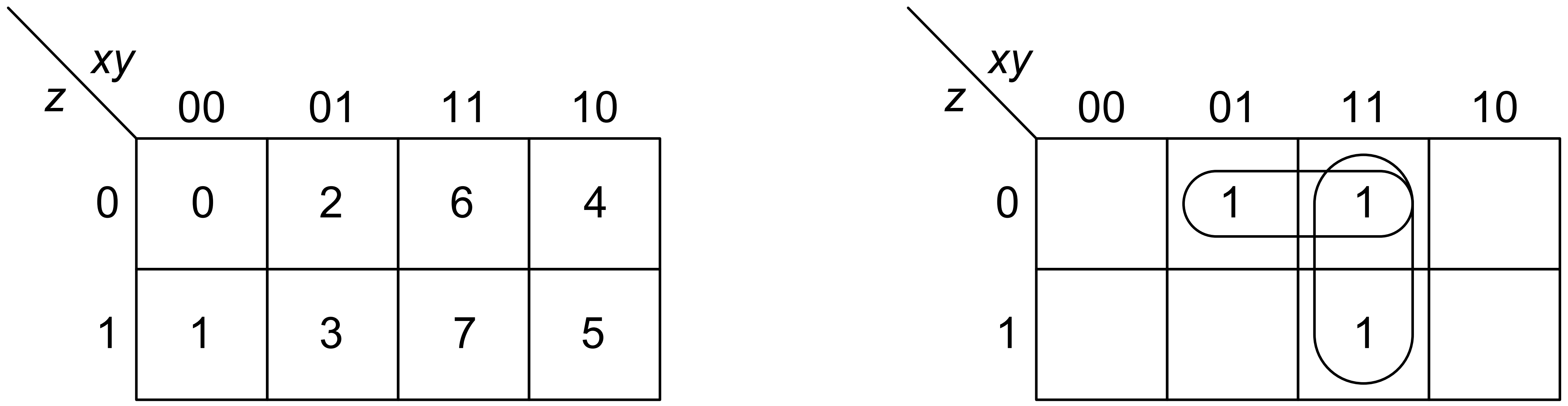


(*c*) Location of minterms in a four-variable map.



(*d*) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

The Map Method



(*a*) Location of minterms in a three-variable map.

(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.

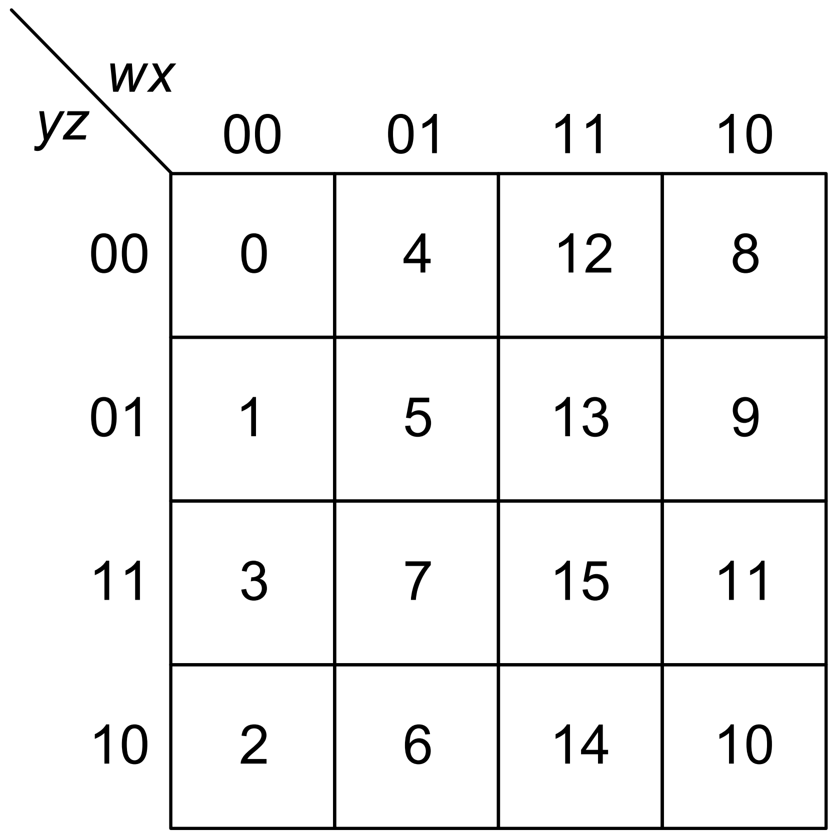
- **Example**: $f = yz' + xy$
  - **Use of cell 6 in forming both cubes justified by idempotent law**
  - Corresponding algebraic manipulations:

$$
\begin{aligned}
f &= x'yz' + xyz' + xyz \\
&= x'yz' + \underline{xyz' + xyz'} + xyz \text{ (idempotent law)} \\
&= yz'(x' + x) + xy(z' + z) \\
&= yz' + xy
\end{aligned}
$$

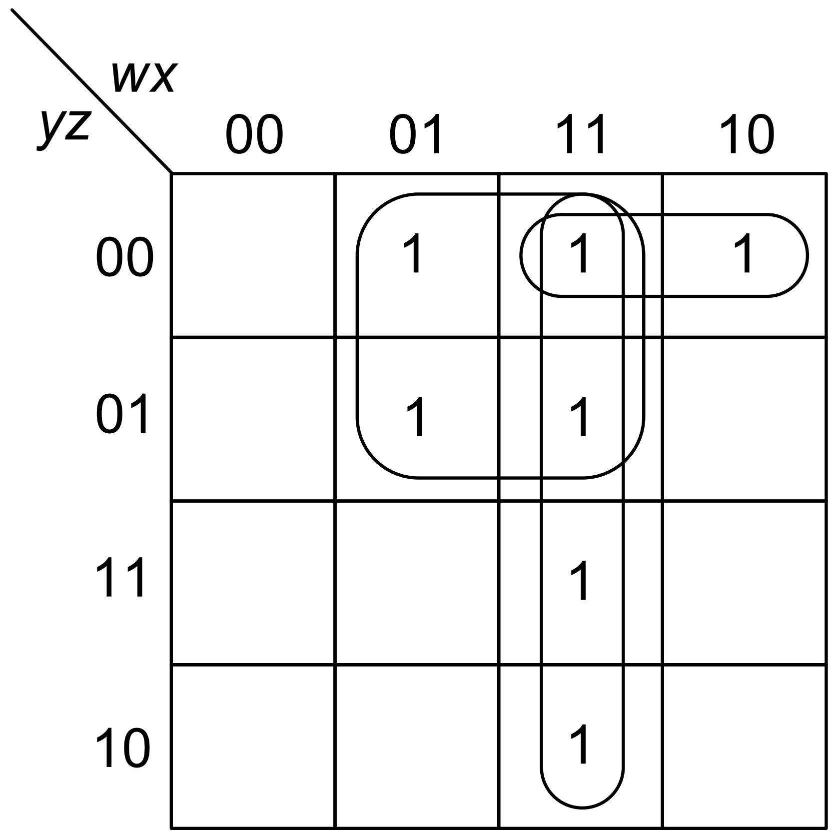
The Map Method

- **Example**: $w'xy'z' + w'xy'z + wxy'z' + wxy'z = xy'(w'z' + w'z + wz' + wz) = xy'$

- **Trick**:
  - In a cube, just keep the variables not changing their value.



(c) Location of minterms in a four-variable map.

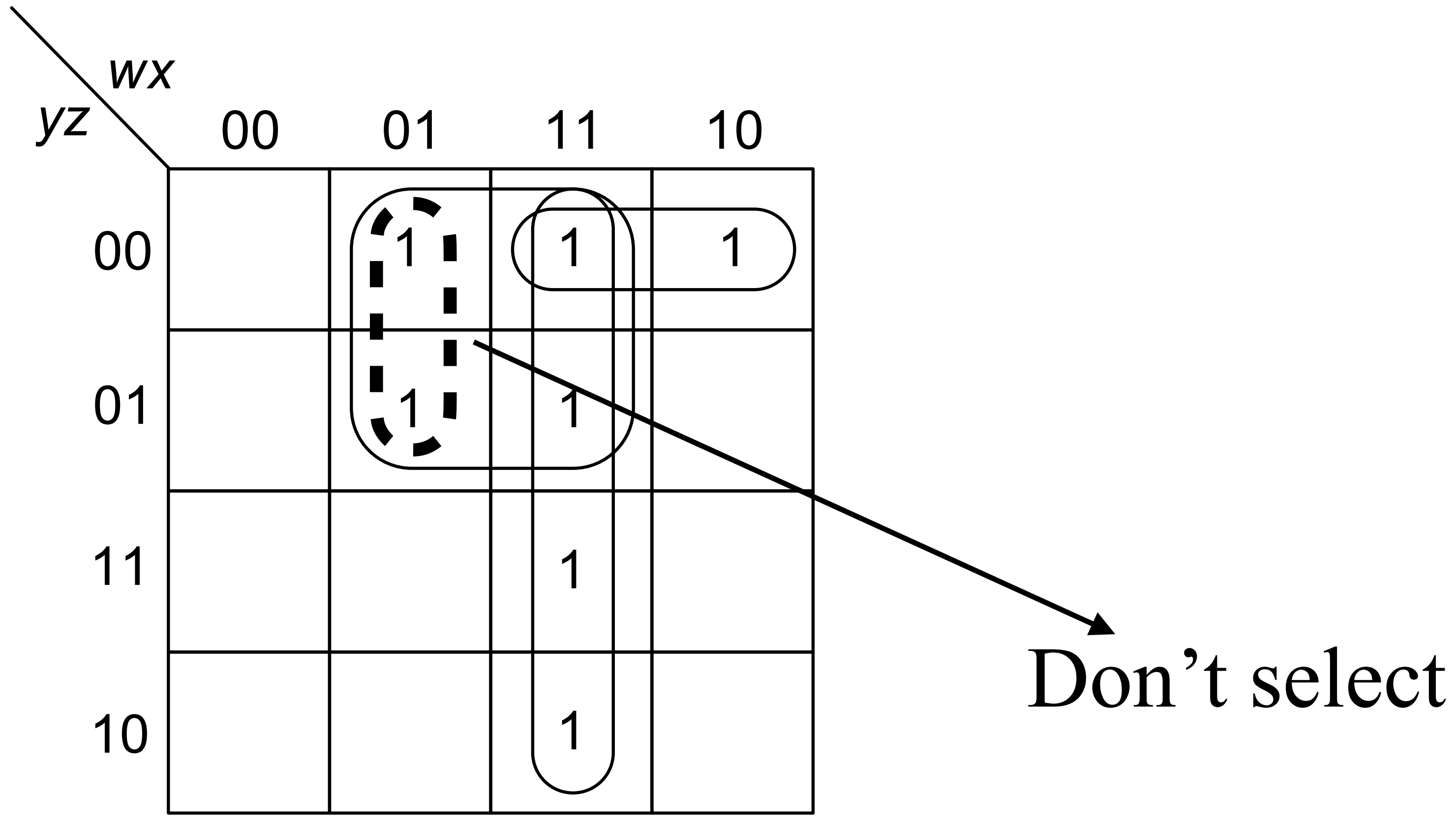


(d) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

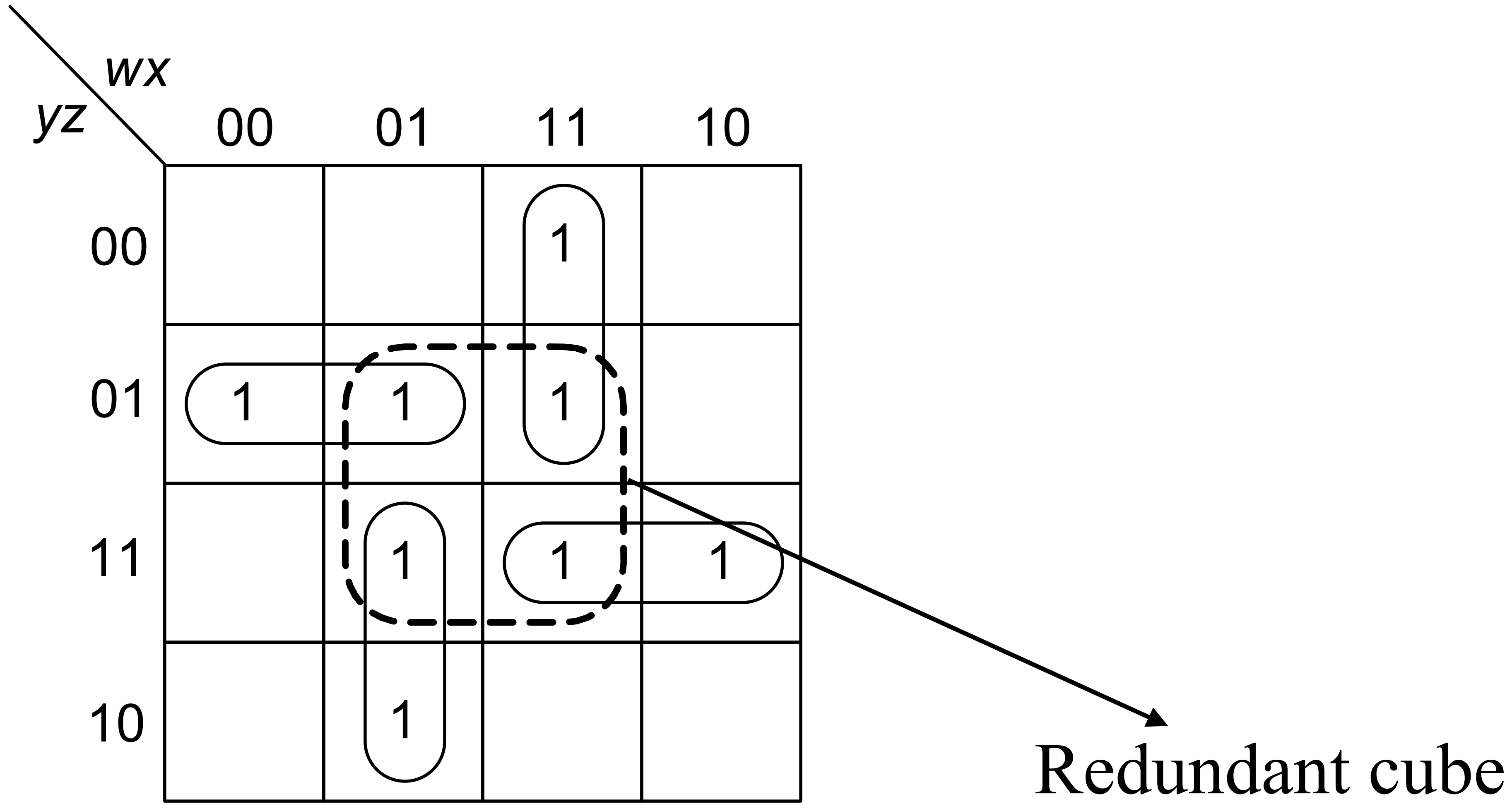
The Map Method

Rules for minimization:

- **Step 1**: cover those 1 cells by cubes that cannot be combined with other 1 cells; continue to 1 cells that have a single adjacent 1 cell (thus can form cubes of only two cells)
- **Step 2—:** Combine 1 cells that yield cubes of four cells, but are not part of any cube of eight cells, and so on..
  - A cube contained in a larger cube must never be selected
  - A cube contained in any combination of other cubes already selected in the cover is redundant (**consensus theorem)**
  - If there are more than one way of covering the map with cubes, select the cover with larger cubes
  - **Minimal expression**: collection of cubes that are as large and as few in number as possible, such that each 1 cell is covered by at least one cube
  - **Irredundant expressions**:
    - An SOP from where no term or literal can be deleted.
    - Not necessarily minimal
  - **Minimal and irredundant expressions may not be unique**
  - **But a minimal expression is always irredundant.**

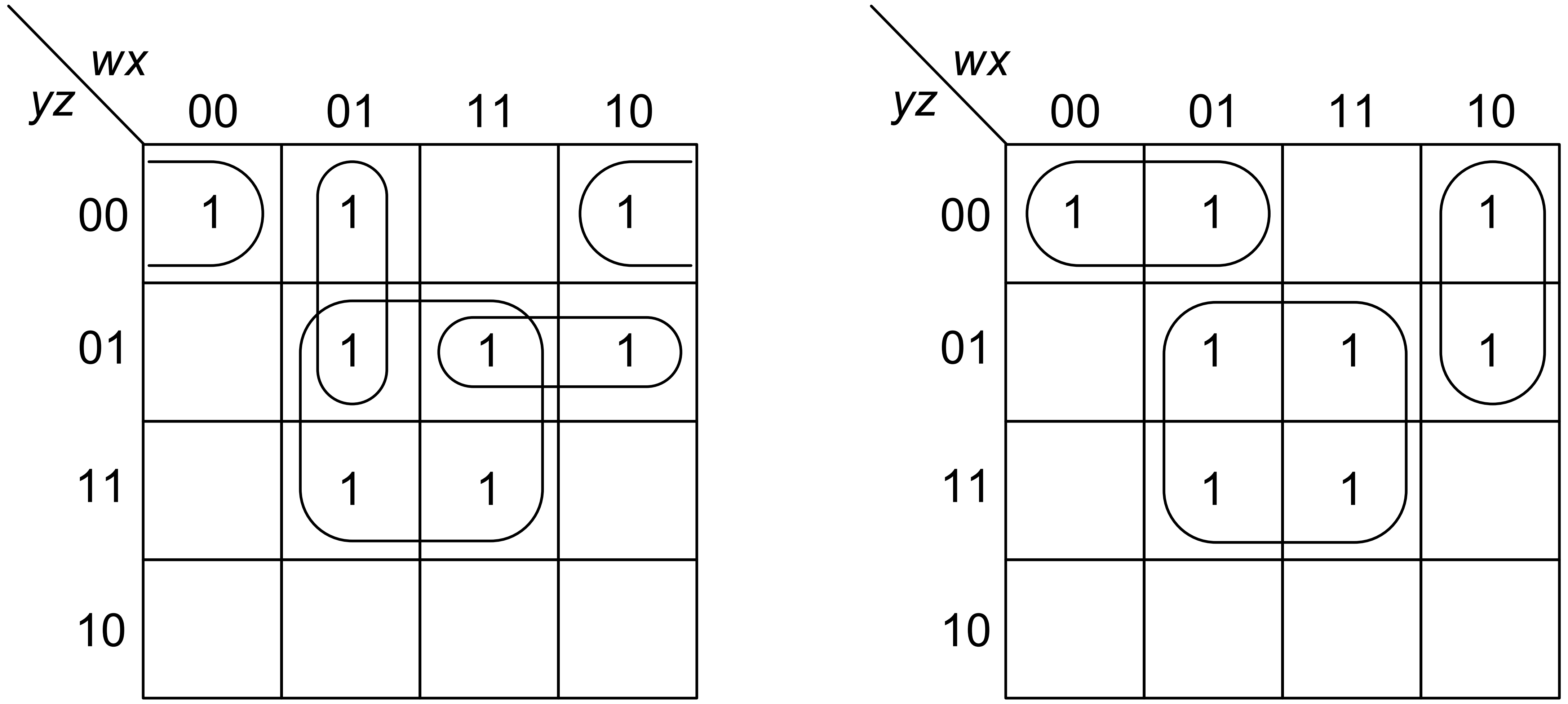


(d) Map for function $f(w,x,y,z)$ $=\sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.



Let's try this..

The Map Method



(*a*) $f = x'y'z' + w'xy' + wy'z + xz$
is an irredundant expression.

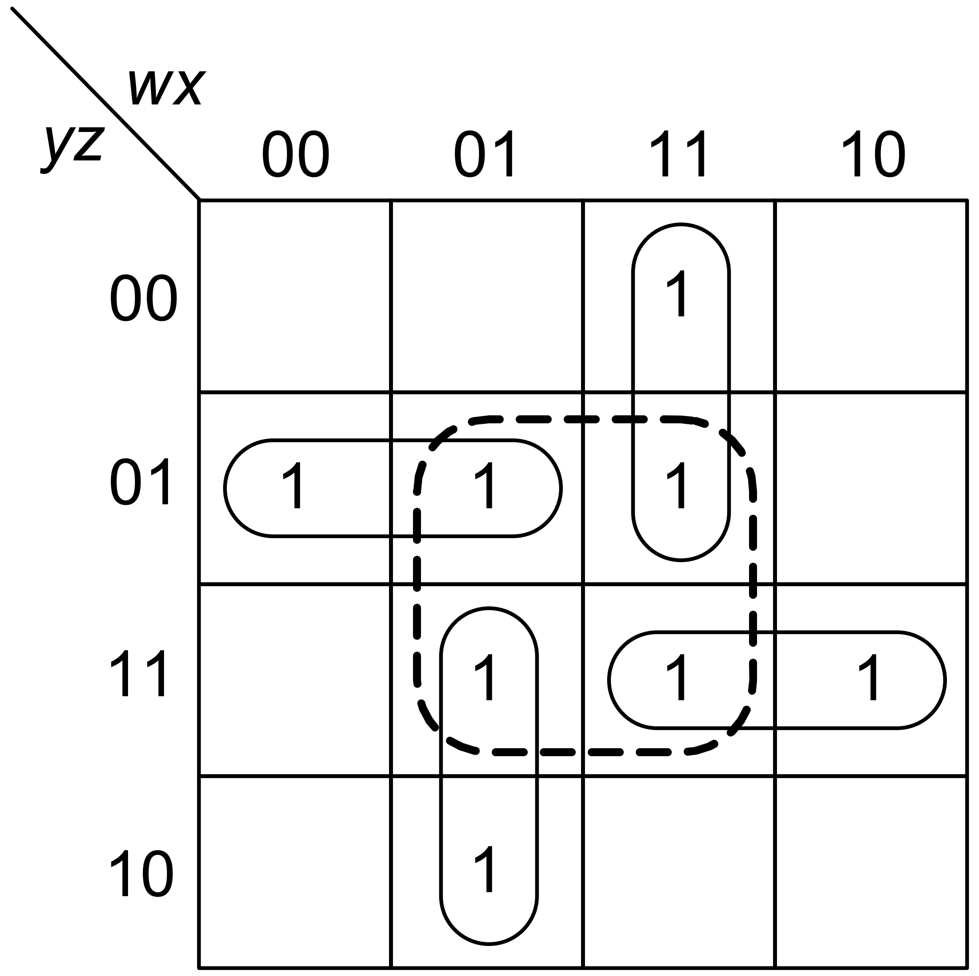
(*b*) $f = w'y'z' + wx'y' + xz$ is the
unique minimal expression.

Example: Two irredundant expressions for $f(w,x,y,z) = \sum(0,4,5,7,8,9,13,15)$

The Map Method

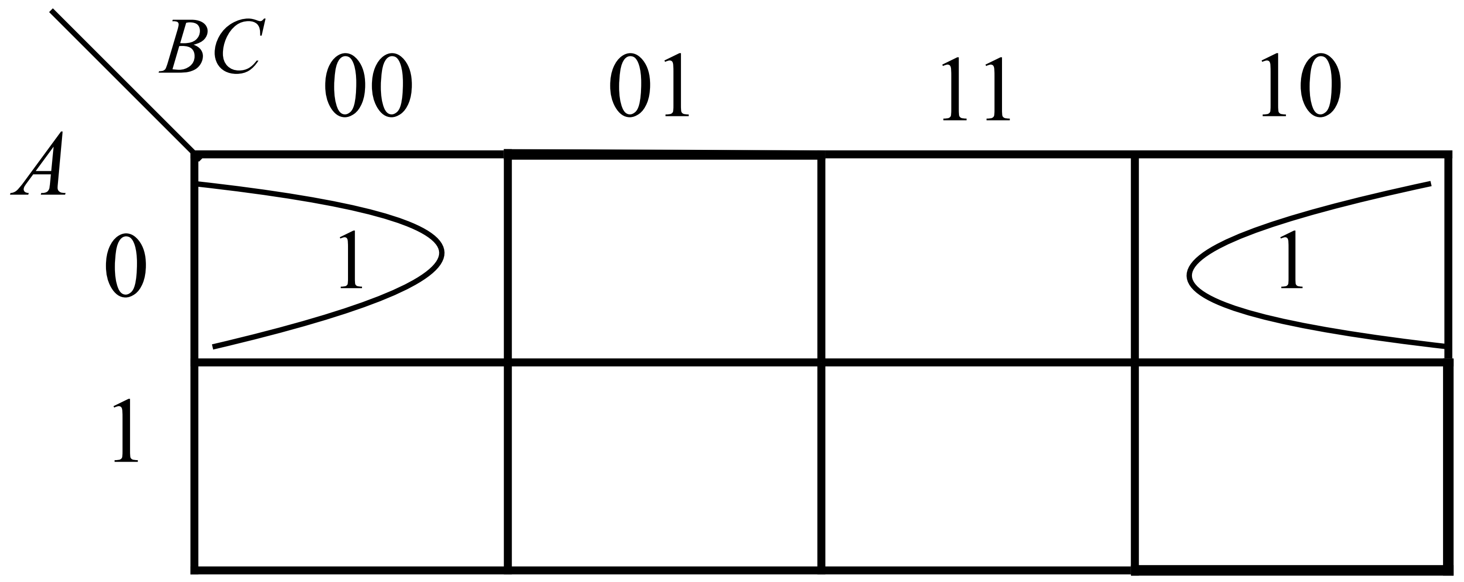
Example: $f(w,x,y,z) = \sum(1,5,6,7,11,12,13,15)$

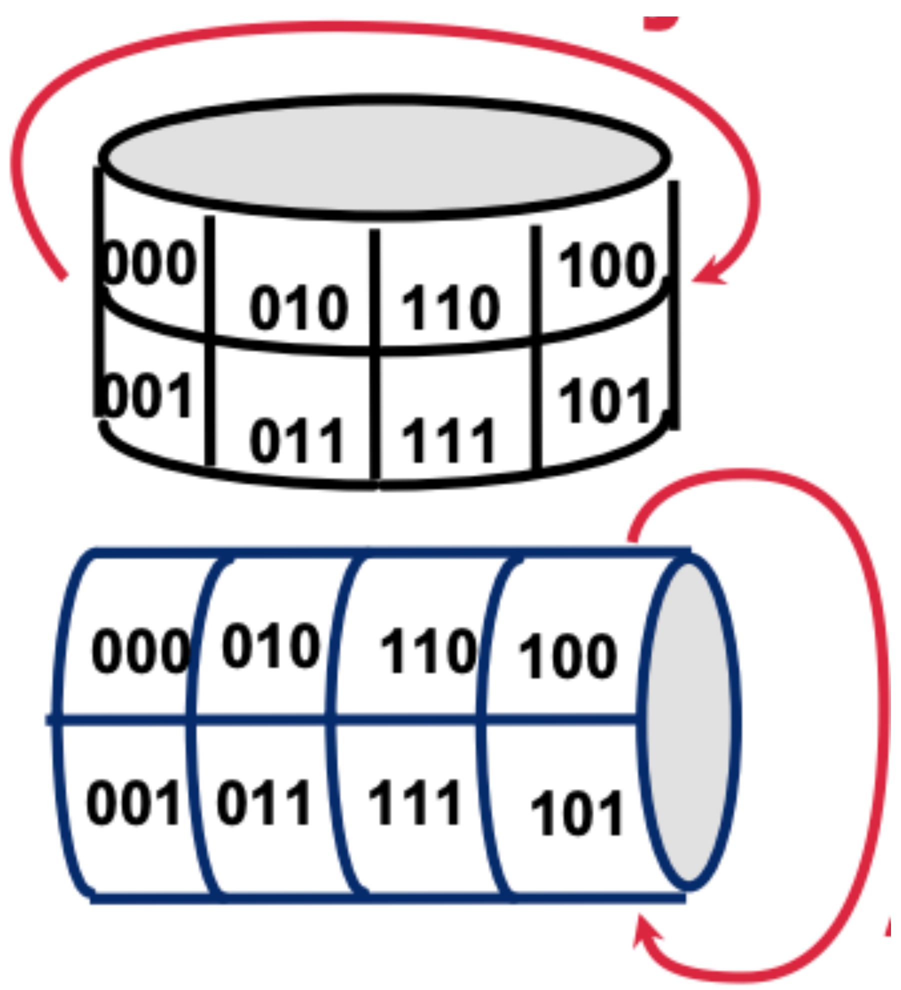
- Only one irredundant form: f = wxy' + wyz + w'xy + w'y'z



The Map Method: Earth is not Flat

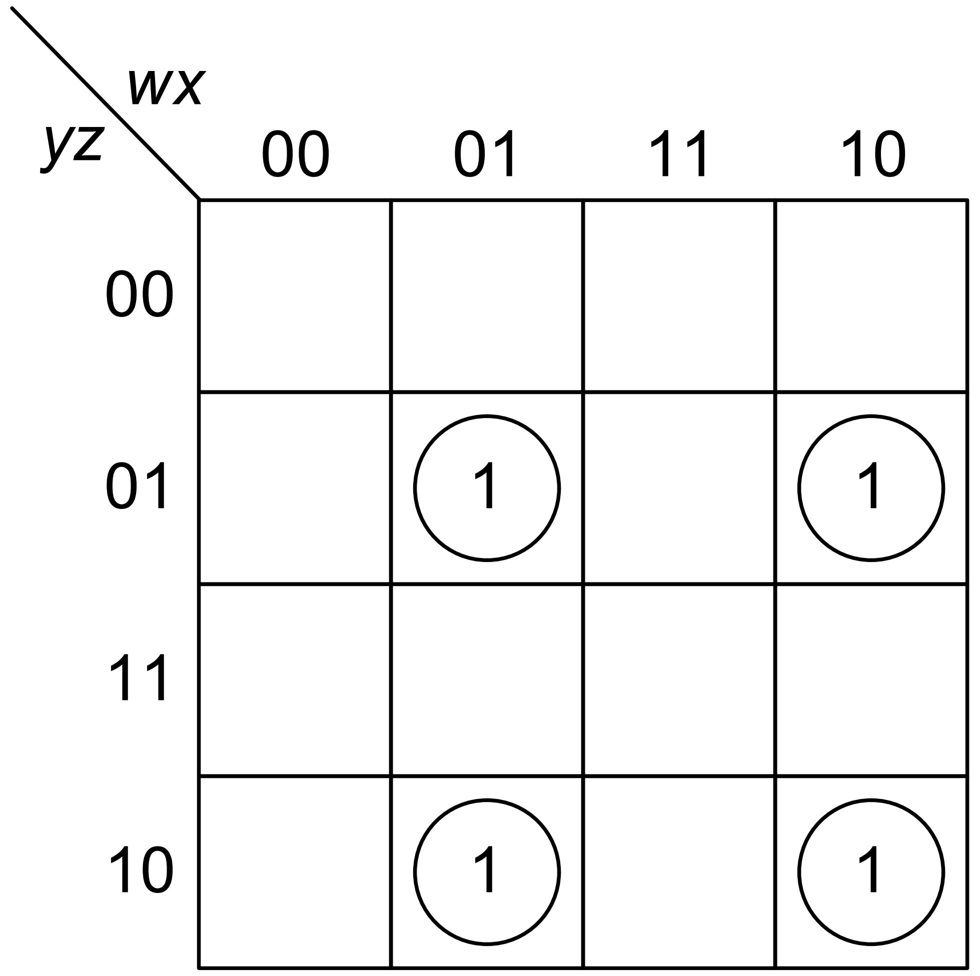
A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110



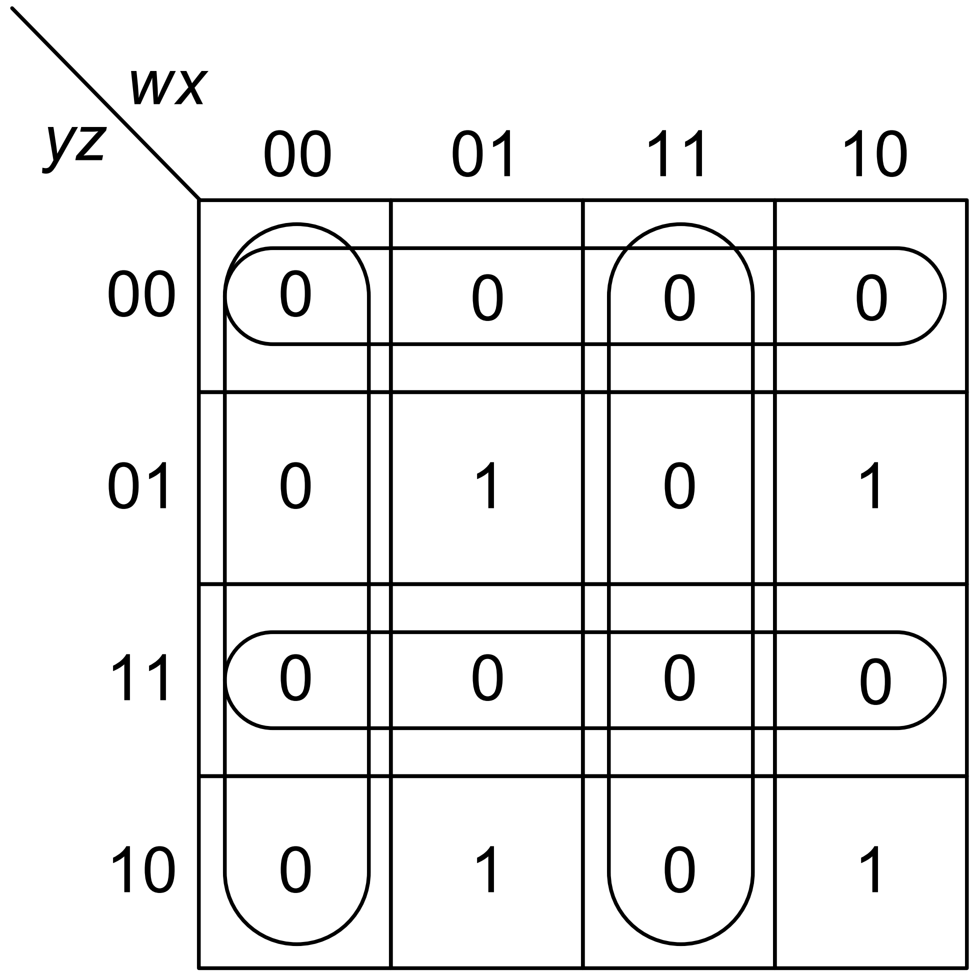


Minimal Product-of-Sums

- **Dual procedure**: product of a minimum number of sum factors, provided there is no other such product with the same number of factors and fewer literals
    - <u>**Variable corresponding to a 1 (0) is complemented (uncomplemented)**</u>
    - Cubes are formed of 0 cells
- **Example**: either one of minimal sum-of-products or minimal product-of-sums can be better than the other in literal count



(a) Map of $f(x,y,z) = \sum(5,6,9,10)$
$= w'xy'z + wx'y'z + w'xyz' + wx'yz'$.



(b) Map of $f(x,y,z)$
$=\prod(0,1,2,3,4,7,8,11,12,13,14,15)$
$= (y + z)(y' + z')(w + x)(w' + x')$.

Let's Try it..

- Implement $f(A, B, C, D) = \sum(0,2,8,12,13)$ with minimum number of gates.

Let's Try it..

- Implement **complement of** $f(A, B, C, D) = \prod(7,9,13)$.

Don’t-care Combinations

- **Don’t-care combination ϕ:** combination for which the value of the function is not specified.
  - Either input combinations may be invalid
  - Or precise output value is of no importance

- Since each don’t-care can be specified as either 0 or 1
  - a function with k don’t-cares corresponds to a class of 2^k distinct functions.
  - Our aim is to choose the function with the minimal representation
- Assign 1 to some don’t-cares and 0 to others in order to increase the size of the selected cubes whenever possible
- No cube containing only don’t-care cells may be formed

Code Converter

Example: code converter from BCD to excess-3 code
Combinations 10 through 15 are don't-cares

Truth table

Decimal number	*BCD inputs* w	x	y	z	*Excess-3 outputs* f_4	f_3	f_2	f_1
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

Code Converter

Example: code converter from BCD to excess-3 code
Combinations 10 through 15 are don't-cares

Truth table

Decimal number	*BCD inputs* w	x	y	z	*Excess-3 outputs* f_4	f_3	f_2	f_1
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

$$f_1 = \sum(0,2,4,6,8) + \sum_\phi(10,11,12,13,14,15)$$
$$f_2 = \sum(0,3,4,7,8) + \sum_\phi(10,11,12,13,14,15)$$
$$f_3 = \sum(1,2,3,4,9) + \sum_\phi(10,11,12,13,14,15)$$
$$f_4 = \sum(5,6,7,8,9) + \sum_\phi(10,11,12,13,14,15)$$

Code Converter

yz \ wx	00	01	11	10
00	1	1	ϕ	1
01			ϕ	
11			ϕ	ϕ
10	1	1	ϕ	ϕ

f_1 map

yz \ wx	00	01	11	10
00	1	1	ϕ	1
01			ϕ	
11	1	1	ϕ	ϕ
10			ϕ	ϕ

f_2 map

yz \ wx	00	01	11	10
00		1	ϕ	
01	1		ϕ	1
11	1		ϕ	ϕ
10	1		ϕ	ϕ

f_3 map

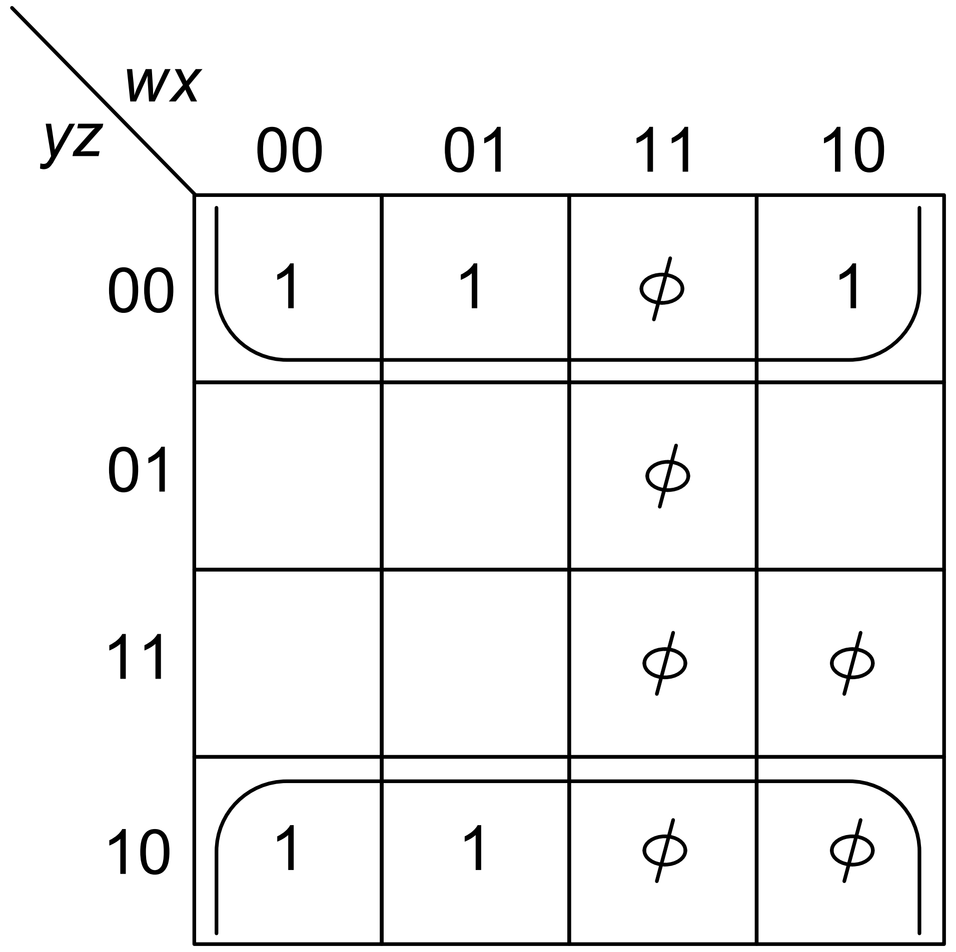
yz \ wx	00	01	11	10
00			ϕ	1
01		1	ϕ	1
11		1	ϕ	ϕ
10		1	ϕ	ϕ

f_4 map

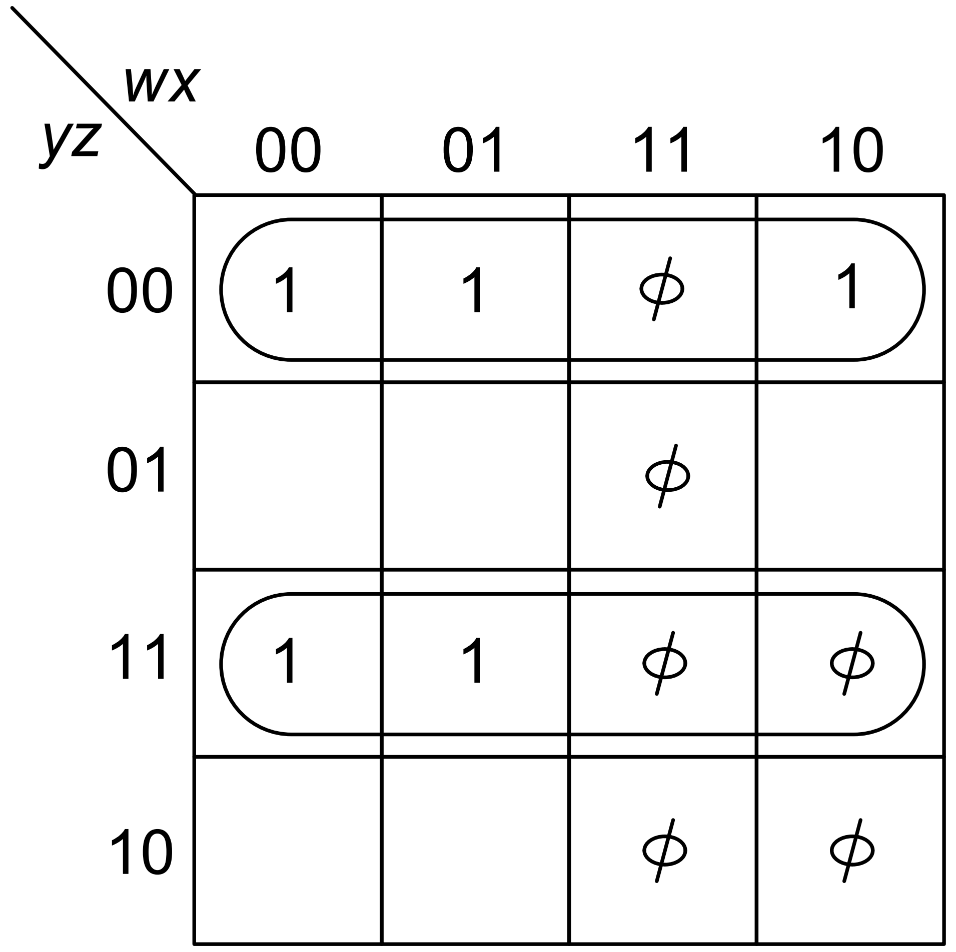
Increase the size of the cubes without making it necessary to increase the number of cubes, than would be required with fewer don’t cares assigned one.

Lets do it!!!

Code Converter

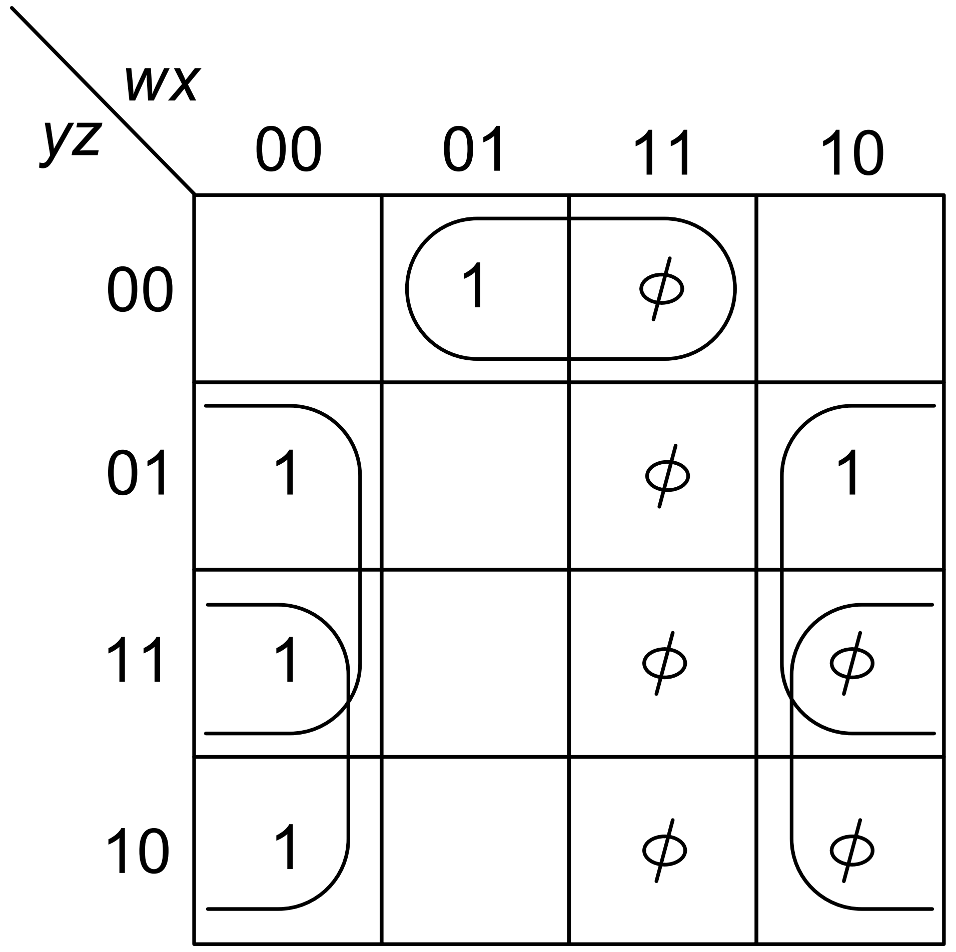


f_1 map

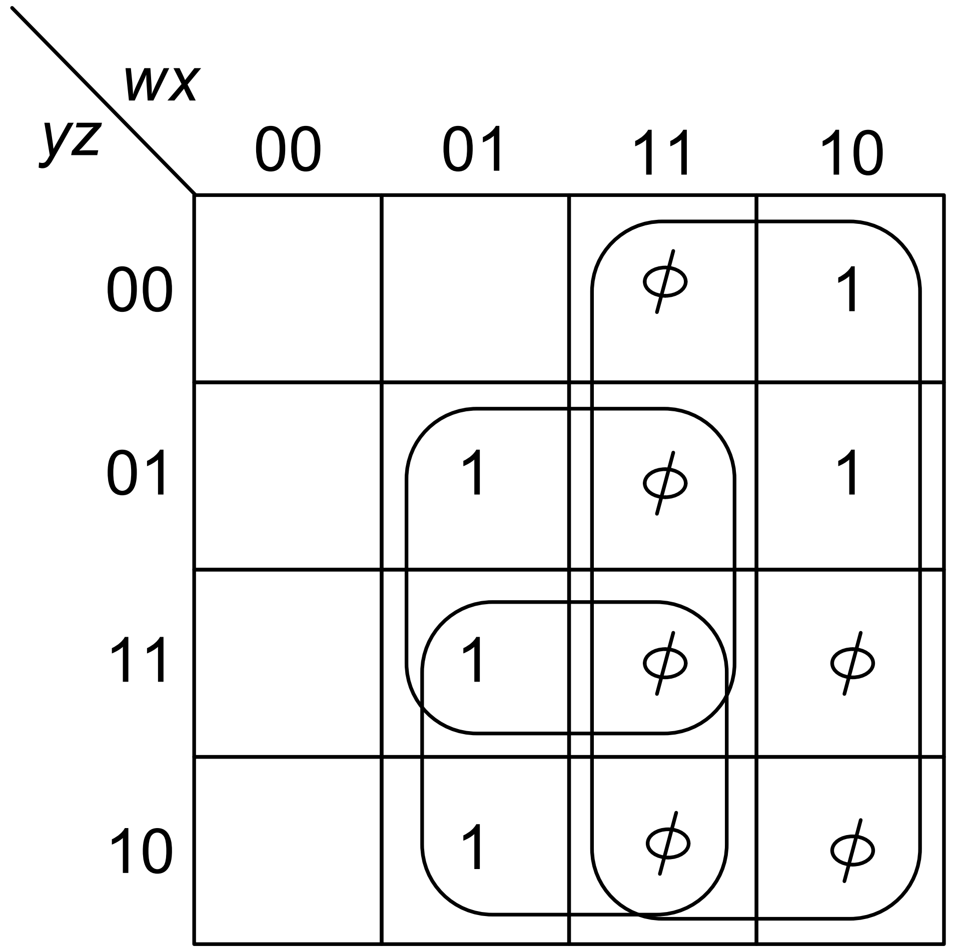


f_2 map

Increase the size of the cubes without making it necessary to increase the number of cubes, than would be required with fewer don't cares assigned one.



f_3 map

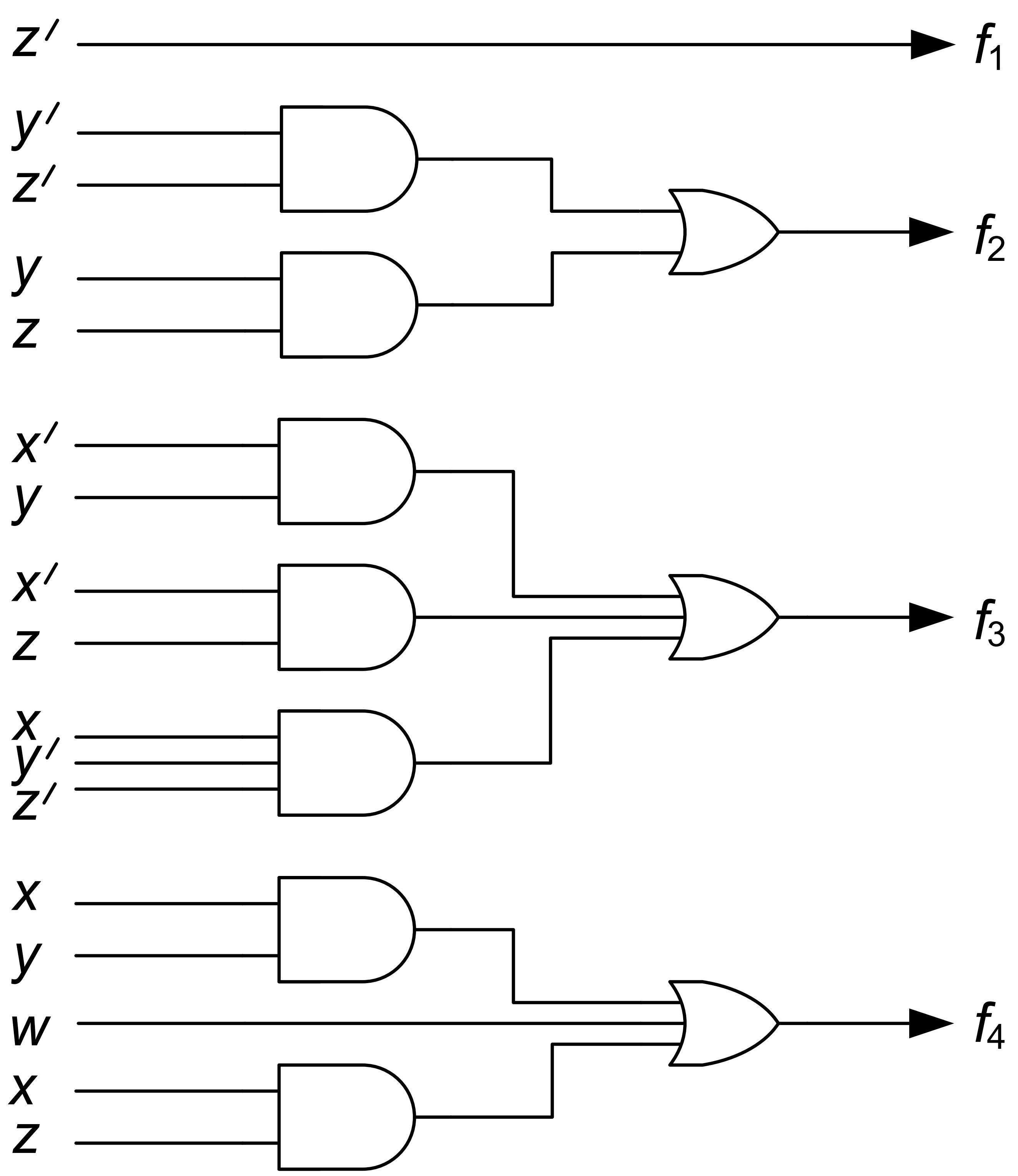


f_4 map

$$f_1 = z'.$$
$$f_2 = y'z' + yz$$
$$f_3 = x'y + x'z + xy'z'$$
$$f_4 = w + xy + xz$$

Logic Network for Code Converter

Two-level AND-OR realization:



Five-variable Map

<u>General five-variable map</u>

yz \ vwx	000	001	011	010	110	111	101	100
00	0	4	12	8	24	28	20	16
01	1	5	13	9	25	29	21	17
11	3	7	15	11	27	31	23	19
10	2	6	14	10	26	30	22	18

Example: Minimize $f(v,w,x,y,z) = \sum(1,2,6,7,9,13,14,15,17,22,23,25,29,30,31)$

000	001	011	010	110	111	101	100
1		1	1	1	1		1
	1	1			1	1	
1	1	1			1	1	

$f(v,w,x,y,z) = x'y'z + wxz + xy + v'w'yz'$

Limitation of Simple Maps

- Maps are useful up to 5-6 variables, after that the calculation becomes formidable..
- How many cells are there in a 6 variable map??
- We also need something which is more amenable to a computer program.